

CS1050 Computer Organization and Digital Design

Nano Processor Design

Group 19

Jayasekara J.P.D.N.R (240271U)
Jackson J.W. (240257H)
Isaiyalan K. (240252M)
Jathursanan S. (240261N)

Contents

1	Lab Task	6
2	High Level Diagram of Nanoprocessor	7
3	Contribution of Team Members	8
4	Basic Nanoprocessor	9
4.1	Instruction Set	9
4.2	Components Overview	9
4.3	Adder 3 Bit	11
4.3.1	Design File	11
4.3.2	Simulation File	12
4.3.3	Elaborated design schematic	13
4.3.4	Timing Diagram	14
4.4	Register Bank	15
4.4.1	Design File	15
4.4.2	Simulation File	17
4.4.3	Elaborated design schematic	19
4.4.4	Timing Diagram	20
4.5	Decoder 3 to 8	21
4.5.1	Design File	21
4.5.2	Simulation File	21
4.5.3	Elaborated design schematic	22
4.5.4	Timing Diagram	22

4.6	Program Counter	23
4.6.1	Design File	23
4.6.2	Simulation File	23
4.6.3	Elaborated design schematic	25
4.6.4	Timing Diagram	25
4.7	Program ROM	26
4.7.1	Design File	26
4.7.2	Simulation File	26
4.7.3	Elaborated design schematic	28
4.7.4	Timing Diagram	28
4.8	Slow Clock	29
4.8.1	Design File	29
4.8.2	Elaborated design schematic	29
4.9	Mux_2_way_3_bit	30
4.9.1	Design File	30
4.9.2	Simulation File	30
4.9.3	Elaborated design schematic	32
4.9.4	Timing Diagram	32
4.10	Mux_2_way_4_bit	33
4.10.1	Design File	33
4.10.2	Simulation File	33
4.10.3	Elaborated design schematic	35
4.10.4	Timing Diagram	35
4.11	Mux_8_way_4_bit	36
4.11.1	Design File	36
4.11.2	Simulation File	36
4.11.3	Elaborated design schematic	39
4.11.4	Timing Diagram	39
4.12	LUT_16_7	41
4.12.1	Design File	41
4.12.2	Elaborated design schematic	41
4.13	Instruction Decoder	42
4.13.1	Design File	42
4.13.2	Simulation File	43
4.13.3	Elaborated design schematic	45
4.13.4	Timing Diagram	46

4.14	ALU	47
4.14.1	Design File	47
4.14.2	Simulation File	47
4.14.3	Elaborated design schematic	49
4.14.4	Timing Diagram	49
4.15	Basic Nanoprocessor	50
4.15.1	Design File	50
4.15.2	Simulation File	54
4.15.3	Elaborated design schematic	56
4.15.4	Timing Diagram	56
4.15.5	Constraint File	56
5	Instructions to Operate Basic Nanoprocessor	58
5.1	Board Pin Mapping	58
5.1.1	Inputs	58
5.1.2	Outputs – LEDs	58
5.1.3	7-Segment Display	58
5.1.4	Anodes	59
5.2	Program (ROM Instructions)	59
5.3	Execution Flow	59
5.4	Final State (R7 = 6)	59
6	Additional Features	60
6.1	4-bit Comparator (Comparator_4_bit)	60
6.2	4-bit Multiplier (Multiplier_4_bit)	60
6.3	Parity Generator (Parity_Generator)	60
6.4	Magnitude Detector (Magnitude_Detector)	60
7	Advanced Nanoprocessor	61
7.1	4-bit Comparator	61
7.1.1	Design File	61
7.1.2	Source File	61
7.1.3	Elaborated design schematic	62
7.1.4	Timing Diagram	63
7.2	4-bit Multiplier	64
7.2.1	Design File	64
7.2.2	Source File	64

7.2.3	Elaborated design schematic	65
7.2.4	Timing Diagram	65
7.3	Parity Generator	66
7.3.1	Design File	66
7.3.2	Source File	66
7.3.3	Elaborated design schematic	67
7.3.4	Timing Diagram	67
7.4	Magnitude Detector	68
7.4.1	Design File	68
7.4.2	Source File	68
7.4.3	Elaborated design schematic	69
7.4.4	Timing Diagram	70
7.5	Instruction Decoder	71
7.5.1	Design File	71
7.5.2	Elaborated design schematic	72
7.6	ALU	73
7.6.1	Design File	73
7.7	Advanced Nanoprocessor	75
7.7.1	Design File	75
7.7.2	Source File	80
7.7.3	Elaborated design schematic	83
7.7.4	Timing Diagram	83
7.7.5	Constraint File	84
8	Instructions to operate Advanced Nanoprocessor	87
8.1	Program (ROM Instructions)	87
8.2	LED Pin Mapping	87
8.3	7-Segment Display	88
8.4	Step-by-Step Demo	88
8.4.1	Step 1 – Reset	88
8.4.2	Step 2 – MOVI R7,3	88
8.4.3	Step 3 – MUL R7,R2	88
8.4.4	Step 4 – ADD R3,R4	88
8.4.5	Step 5 – NEG R7	88
8.5	LED Pattern Summary	89
8.6	Feature Summary	89

8.6.1	Comparator	89
8.6.2	Multiplier	89
8.6.3	Parity Generator	89
8.6.4	Magnitude Detector	89
9	Problems Faced and Solutions	90
9.1	Incorrect Load_sel Logic in Instruction Decoder	90
9.2	Slow Clock Too Slow for Simulation	90
9.3	DRC Errors NSTD-1 and UCIO-1 (Advanced Version)	91
10	Resource Optimizations	91
10.1	with-select for 2-Way Multiplexers	91
10.2	case-when for 8-Way Multiplexer	91
10.3	to_integer Indexing for 3-to-8 Decoder	92

1 Lab Task

The lab task is to design, implement, and verify a simple 4-bit nanoprocessor capable of executing a limited set of instructions. This includes developing essential components such as a 4-bit add/subtract unit that supports signed arithmetic using 2's complement representation, a 3-bit program counter with reset functionality, and a register bank consisting of eight 4-bit registers where register R0 is permanently set to zero. In addition, we must design and integrate k-way b-bit multiplexers (or equivalent tri-state bus systems) to manage data flow between components, as well as a 3-bit adder to increment the program counter. A key part of the design is the instruction decoder, which interprets 12-bit instructions and generates appropriate control signals to activate only the required components during execution.

Furthermore, a program ROM must be created to store the machine code of an assembly program written using the given instruction set (MOVI, ADD, NEG, and JZR). The complete system should be connected using appropriate buses and driven by a slow clock to allow step-by-step observation of execution. We are required to simulate individual components and the full system before implementing the design on the BASYS 3 FPGA board. The processor must successfully execute a sample program, such as calculating the sum of integers from 1 to 3, with the final result stored in register R7 and displayed using LEDs and a 7-segment display. The lab also emphasizes teamwork, requiring students to divide tasks, integrate components, and collectively demonstrate and explain their design.

2 High Level Diagram of Nanoprocessor

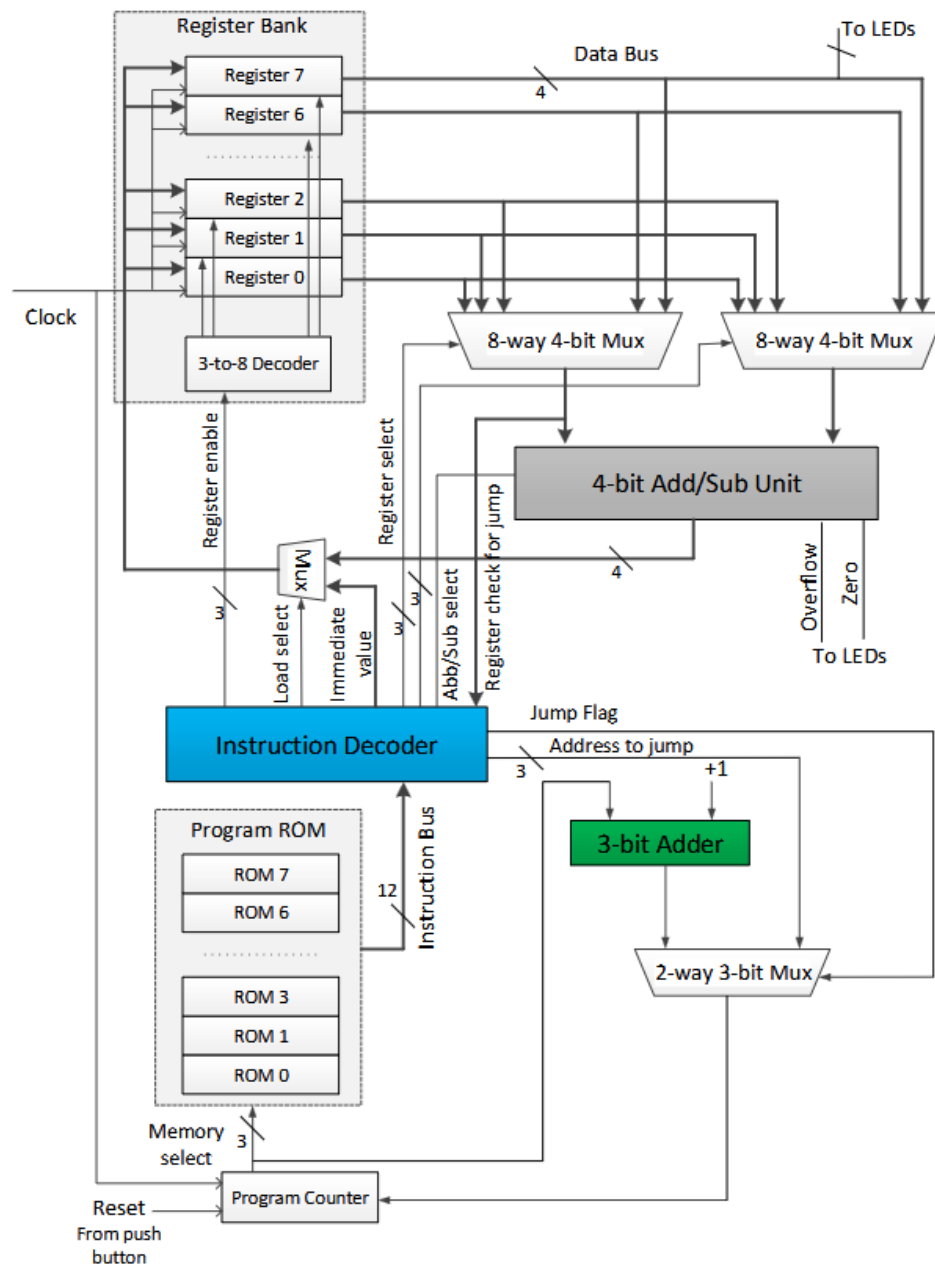


Figure 1: High Level Diagram of Nanoprocessor (Source: Lab Instruction File)

3 Contribution of Team Members

Team Member	Contribution	Time Spent
Jayasekara J.P.D.N.R.	Designed the Instruction Decoder and generated all control signals including register enable, MUX selection, ALU operation control, and jump logic. Implemented the Program Counter, 3-bit adder (PC+1), and Program ROM for instruction storage and execution control.	20 hours
Jackson J.W.	Designed all multiplexers including 8-way 4-bit MUX, 2-way 4-bit MUX, and 2-way 3-bit MUX. Developed 4-bit, 3-bit, and 12-bit bus structures. Managed data routing between registers, ALU, and immediate values ensuring correct system data flow.	19 hours
Isaiyalan K.	Designed and implemented the 4-bit Add/Subtract Unit, including 2's complement logic for subtraction and NEG operations. Handled overflow detection and supported testing of ADD and NEG instructions.	18 hours
Jathursanan S.	Implemented an 8×4 -bit register bank (R0–R7) with R0 hardwired to zero. Designed the 3-to-8 decoder for register selection, register enable logic, and reset functionality. Ensured correct data storage and retrieval across the system.	18 hours
Together	The nanoprocessor was developed as a fully integrated system combining arithmetic logic, data storage, data routing, and control units. Each module was independently designed and then integrated to form a working 4-bit processor capable of executing a small instruction set. Extensive teamwork was involved in debugging and verification using simulation and hardware testing on the BASYS 3 board. The final system successfully executed instructions such as MOV and ADD, with outputs displayed via LEDs and a 7-segment display. The project improved understanding of processor architecture, modular design, and hardware-software integration.	7 hours

4 Basic Nanoprocessor

4.1 Instruction Set

Questions	Assembly Instruction	Machine Code
Calculation of total between the numbers 1 to 3	MOV R7, 1	101110000001
	MOV R5, 2	101010000010
	MOV R4, 3	101000000011
	ADD R7, R5	001111010000
	ADD R7, R4	001111000000

4.2 Components Overview

Component	Function	Structure
4-bit Adder / Subtractor	It will add or subtract 4-bit inputs. (Subtraction is done by adding two's complement of second input). This circuit can be used to negate numbers. It has flags to indicate if there is an overflow in the process or if the output is zero.	It has 4 Full Adders in it. It has an input to select whether the circuit acts as an Adder or Subtractor.
3-bit Adder	The main function of the 3-bit adder is incrementing the current address stored in the program counter one by one.	It is implemented by using three full adders.
3-bit Program Counter	It provides the address to access the next instruction to be executed.	It is implemented by using 3 D-Flip Flops, a Clock signal, and a Reset option.
Register Bank	4-bit data can be stored in the registers. The register can be selected through the control input.	It has 8 4-bit registers in it. A 3 to 8 Decoder is used to select a specific register in which the data should be stored.
Program ROM	In each ROM, 12-bit instructions (assembly program) are stored. The 13 bits are divided into various fields such as opcode and operand.	It has 8 ROMs inside. Memory select selects which program has to be executed and the instruction is fed to the instruction decoder.
2-way 3-bit Multiplexer	Used in ROM incrementer for choosing one of the 3-bit values (program jump address and 3-bit adder's output).	It is implemented using a process block with a case statement that checks the Jump signal; if it is 0, it passes the input, otherwise it passes output Q.

Component	Function	Structure
2-way 4-bit Multi-plexer	Used for choosing one of the values from immediate value and output from add/sub unit.	It is implemented using a process block with a case statement that checks value of jump; if it is 0, it assigns value to Q otherwise it assigns output to Q.
8-way 4-bit Multi-plexer	It chooses one of 8 registers and reads data from that register and sends it to ALU or program decoder.	It is implemented using a process block with a case statement that selects value of Reg Sel signal across R0 to R7 register to output Q.
Instruction Decoder	Decodes instruction fetched from program ROM and activates necessary units.	It is a 2 to 4 Decoder to decode the first two bits (MSB side).
LUT	The main function of LUT is to convert a 4-bit binary input (data) into its corresponding 7-segment display encoding which visually represents hexadecimal digits (0-F).	It is implemented using a ROM-like lookup table (array of 16 pre-defined 7-bit patterns) and a signal assignment that maps the 4-bit input to the corresponding pattern.
Slow Clock	The main function of slow clock is to divide a high-frequency clock into a slower clock signal by counting clock cycles.	It is implemented using a 26-bit counter that increments on every rising edge of Clk. The output clock is derived from one specific bit of the counter, effectively slowing down the frequency.

4.3 Adder 3 Bit

4.3.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Adder_3_Bit is
    Port ( A : in STD_LOGIC_VECTOR (2 downto 0);
          C_in : in STD_LOGIC;
          S : out STD_LOGIC_VECTOR (2 downto 0);
          C_out : out STD_LOGIC);
end Adder_3_Bit;

architecture Behavioral of Adder_3_Bit is

component FA
    port(
        A : in std_logic;
        B : in std_logic;
        C_in : in std_logic;
        S : out std_logic;
        C_out : out std_logic);
end component;

SIGNAL FA0_S, FA0_C, FA1_S, FA1_C, FA2_S : std_logic;
SIGNAL B : STD_LOGIC_VECTOR (2 downto 0) := "001";

begin

    FA_0 : FA
        port map (
            A => A(0),
            B => B(0),
            C_in => C_in,
            S => S(0),
            C_out => FA0_C);

    FA_1 : FA
        port map (
            A => A(1),
            B => B(1),
            C_in => FA0_C,
            S => S(1),
            C_out => FA1_C);

    FA_2 : FA
        port map (
            A => A(2),
            B => B(2),
            C_in => FA1_C,
            S => S(2),
            C_out => C_out);

end Behavioral;
```

4.3.2 Simulation File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Adder_3_Bit is
end TB_Adder_3_Bit;

architecture Behavioral of TB_Adder_3_Bit is

    component Adder_3_Bit
        Port ( A      : in  STD_LOGIC_VECTOR (2 downto 0);
              C_in   : in  STD_LOGIC;
              S      : out STD_LOGIC_VECTOR (2 downto 0);
              C_out  : out STD_LOGIC);
    end component;

    signal A_tb      : STD_LOGIC_VECTOR(2 downto 0) := (others => '0');
    signal C_in_tb   : STD_LOGIC := '0';
    signal S_tb      : STD_LOGIC_VECTOR(2 downto 0);
    signal C_out_tb  : STD_LOGIC;

begin

    uut: Adder_3_Bit port map (
        A      => A_tb,
        C_in   => C_in_tb,
        S      => S_tb,
        C_out  => C_out_tb
    );

    stim_proc: process
    begin

        A_tb <= "000";
        C_in_tb <= '0';
        wait for 100 ns;

        A_tb <= "001";
        C_in_tb <= '0';
        wait for 100 ns;

        A_tb <= "010";
        C_in_tb <= '0';
        wait for 100 ns;

        A_tb <= "011";
        C_in_tb <= '0';
        wait for 100 ns;

        A_tb <= "100";
        C_in_tb <= '0';
        wait for 100 ns;

        A_tb <= "101";
        C_in_tb <= '0';
        wait for 100 ns;
```

```

    A_tb <= "110";
    C_in_tb <= '0';
    wait for 100 ns;

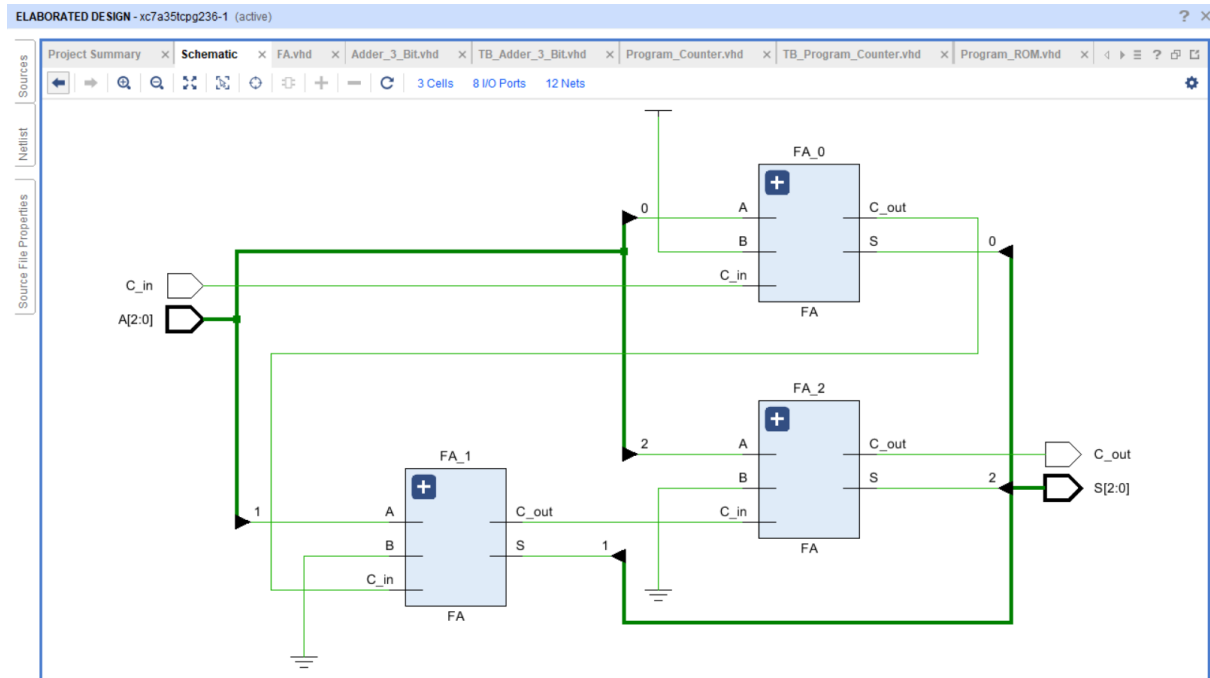
    A_tb <= "111";
    C_in_tb <= '0';

    wait;
end process;

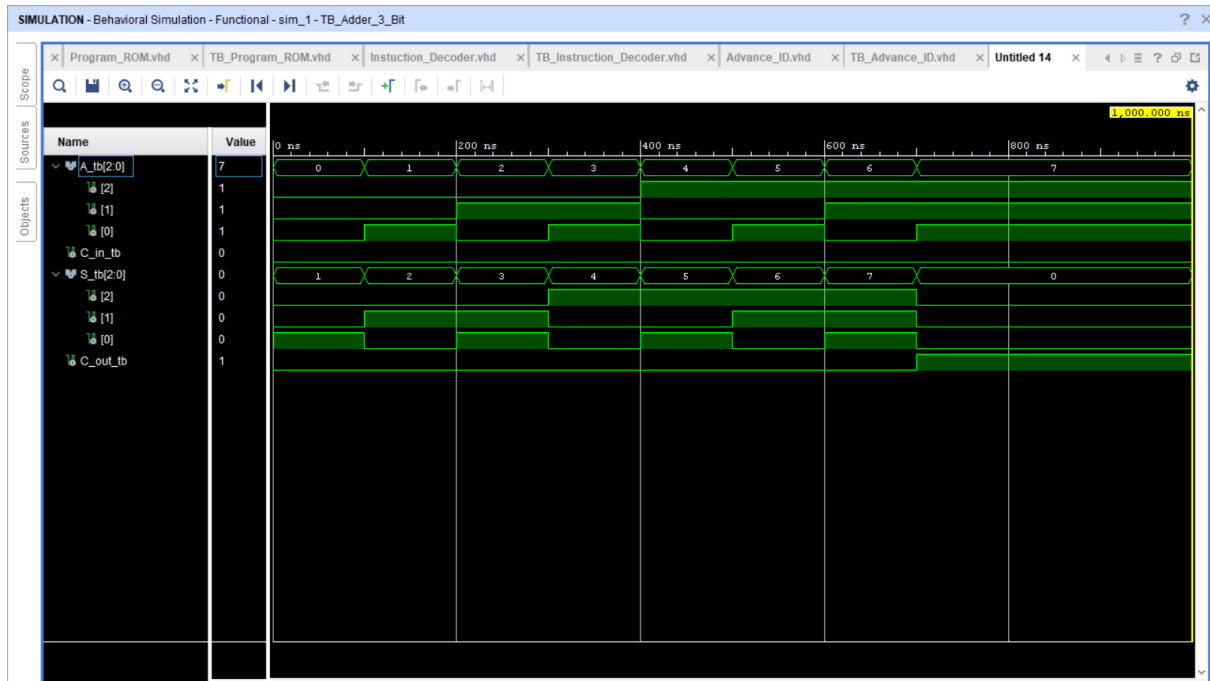
end Behavioral;

```

4.3.3 Elaborated design schematic



4.3.4 Timing Diagram



4.4 Register Bank

4.4.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Register_Bank is
    Port ( Reg_en : in  STD_LOGIC_VECTOR (2 downto 0);
          Clk     : in  STD_LOGIC;
          Reset   : in  STD_LOGIC;
          Inp     : in  STD_LOGIC_VECTOR (3 downto 0);
          Q0      : out STD_LOGIC_VECTOR (3 downto 0);
          Q1      : out STD_LOGIC_VECTOR (3 downto 0);
          Q2      : out STD_LOGIC_VECTOR (3 downto 0);
          Q3      : out STD_LOGIC_VECTOR (3 downto 0);
          Q4      : out STD_LOGIC_VECTOR (3 downto 0);
          Q5      : out STD_LOGIC_VECTOR (3 downto 0);
          Q6      : out STD_LOGIC_VECTOR (3 downto 0);
          Q7      : out STD_LOGIC_VECTOR (3 downto 0));
end Register_Bank;

architecture Behavioral of Register_Bank is

    component Decoder_3_to_8
        Port ( I : in  STD_LOGIC_VECTOR (2 downto 0);
              EN : in  STD_LOGIC;
              Y : out STD_LOGIC_VECTOR (7 downto 0));
    end component;

    component Reg
        Port ( D      : in  STD_LOGIC_VECTOR (3 downto 0);
              En      : in  STD_LOGIC;
              Clk     : in  STD_LOGIC;
              Reset   : in  STD_LOGIC;
              Q       : out STD_LOGIC_VECTOR (3 downto 0));
    end component;

    signal sel : STD_LOGIC_VECTOR (7 downto 0);

begin

    Decoder_3_to_8_0 : Decoder_3_to_8
        PORT MAP (
            I  => Reg_en,
            EN => '1',
            Y  => sel);

    R0 : Reg
        PORT MAP (
            D      => "0000",
            En     => sel(0),
            Clk    => Clk,
            Reset  => Reset,
            Q      => Q0);

    R1 : Reg
        PORT MAP (
```

```

        D      => Inp,
        En     => sel(1),
        Clk    => Clk,
        Reset  => Reset,
        Q      => Q1);

R2 : Reg
    PORT MAP (
        D      => Inp,
        En     => sel(2),
        Clk    => Clk,
        Reset  => Reset,
        Q      => Q2);

R3 : Reg
    PORT MAP (
        D      => Inp,
        En     => sel(3),
        Clk    => Clk,
        Reset  => Reset,
        Q      => Q3);

R4 : Reg
    PORT MAP (
        D      => Inp,
        En     => sel(4),
        Clk    => Clk,
        Reset  => Reset,
        Q      => Q4);

R5 : Reg
    PORT MAP (
        D      => Inp,
        En     => sel(5),
        Clk    => Clk,
        Reset  => Reset,
        Q      => Q5);

R6 : Reg
    PORT MAP (
        D      => Inp,
        En     => sel(6),
        Clk    => Clk,
        Reset  => Reset,
        Q      => Q6);

R7 : Reg
    PORT MAP (
        D      => Inp,
        En     => sel(7),
        Clk    => Clk,
        Reset  => Reset,
        Q      => Q7);

end Behavioral;
```

4.4.2 Simulation File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity TB_Register_Bank is
end TB_Register_Bank;

architecture Behavioral of TB_Register_Bank is
    component Register_Bank is
    Port (
        Reg_en: in STD_LOGIC_VECTOR (2 downto 0);
        Clk: in STD_LOGIC;
        Reset: in STD_LOGIC;
        Inp: in STD_LOGIC_VECTOR (3 downto 0);
        Q0 : out STD_LOGIC_VECTOR (3 downto 0);
        Q1 : out STD_LOGIC_VECTOR (3 downto 0);
        Q2 : out STD_LOGIC_VECTOR (3 downto 0);
        Q3 : out STD_LOGIC_VECTOR (3 downto 0);
        Q4 : out STD_LOGIC_VECTOR (3 downto 0);
        Q5 : out STD_LOGIC_VECTOR (3 downto 0);
        Q6 : out STD_LOGIC_VECTOR (3 downto 0);
        Q7 : out STD_LOGIC_VECTOR (3 downto 0)
    );
    end component;

    signal Reg_en_TB : STD_LOGIC_VECTOR (2 downto 0);
    signal Clk_TB, Reset_TB : STD_LOGIC := '0';
    signal Inp_TB : STD_LOGIC_VECTOR (3 downto 0) := "0000";
    signal Q0_TB, Q1_TB, Q2_TB, Q3_TB, Q4_TB, Q5_TB, Q6_TB, Q7_TB :
        STD_LOGIC_VECTOR (3 downto 0);

begin

    UUT : Register_Bank
        port map (
            Reg_en => Reg_en_TB,
            Clk     => Clk_TB,
            Reset   => Reset_TB,
            Inp     => Inp_TB,
            Q0      => Q0_TB,
            Q1      => Q1_TB,
            Q2      => Q2_TB,
            Q3      => Q3_TB,
            Q4      => Q4_TB,
            Q5      => Q5_TB,
            Q6      => Q6_TB,
            Q7      => Q7_TB
        );

    process
    begin
        Reg_en_TB <= "000";
        Inp_TB    <= "0000";
        wait for 100 ns;

        Reg_en_TB <= "001";
```

```

    Inp_TB    <= "1111";
    wait for 100 ns;

    Reg_en_TB <= "010";
    Inp_TB    <= "1010";
    wait for 100 ns;

    Reg_en_TB <= "100";
    Inp_TB    <= "0101";
    wait for 100 ns;

    Reg_en_TB <= "101";
    Inp_TB    <= "1111";
    wait for 100 ns;

    Reg_en_TB <= "111";
    Inp_TB    <= "1100";
    wait for 100 ns;

    wait;
end process;

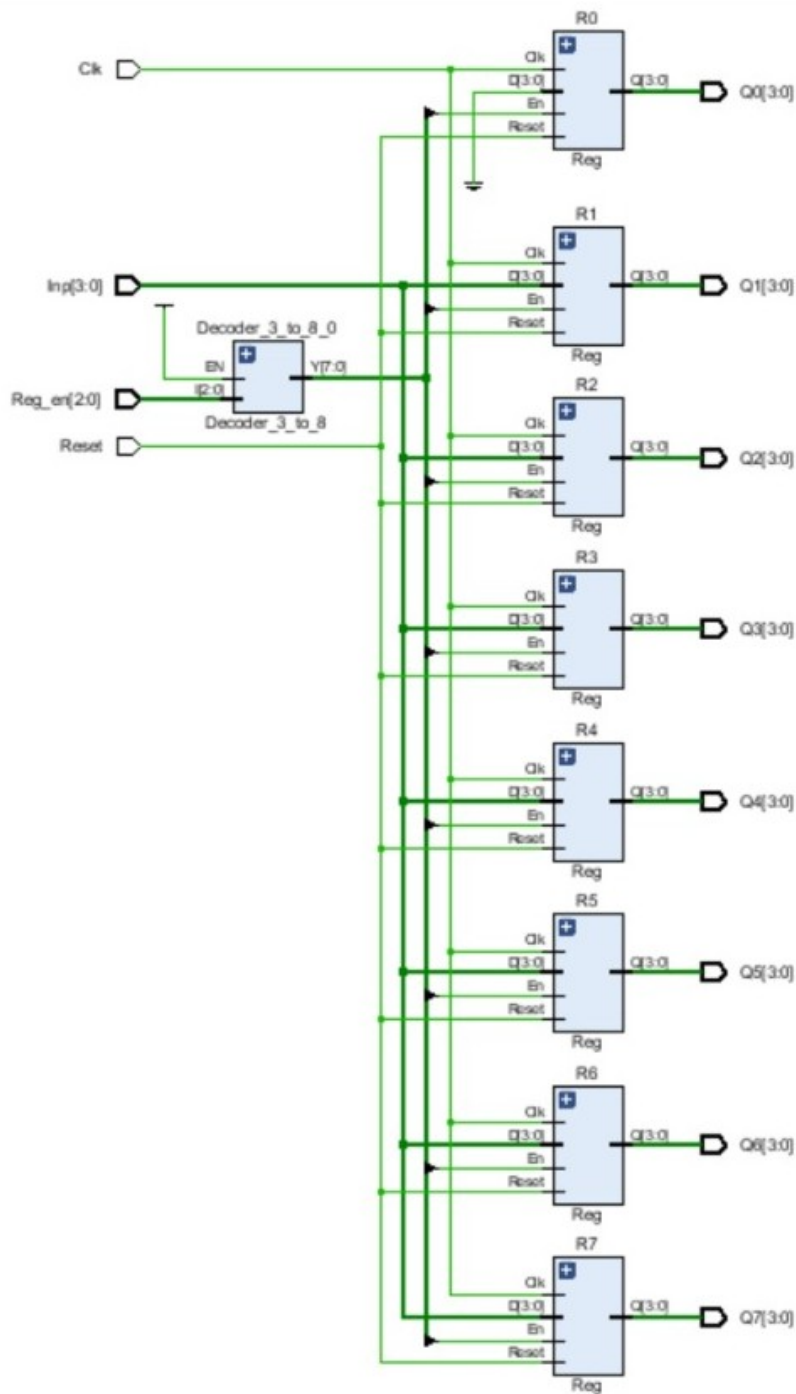
process
begin
    Clk_TB <= (not Clk_TB);
    wait for 5ns;
end process;

process
begin
    Reset_TB <= '0';
    wait;
end process;

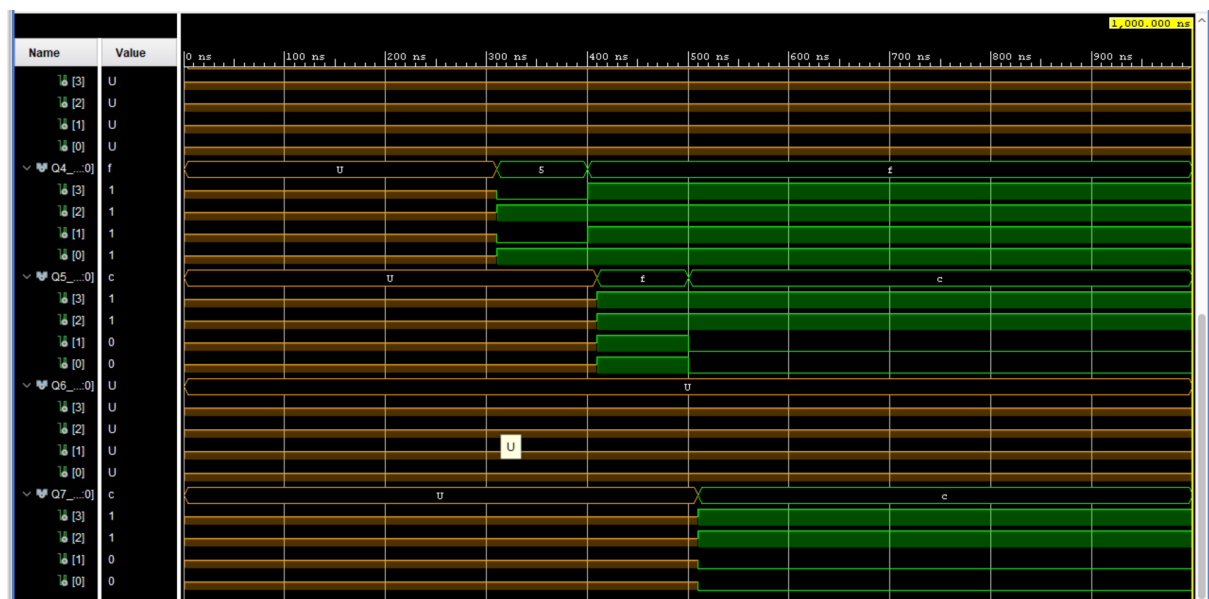
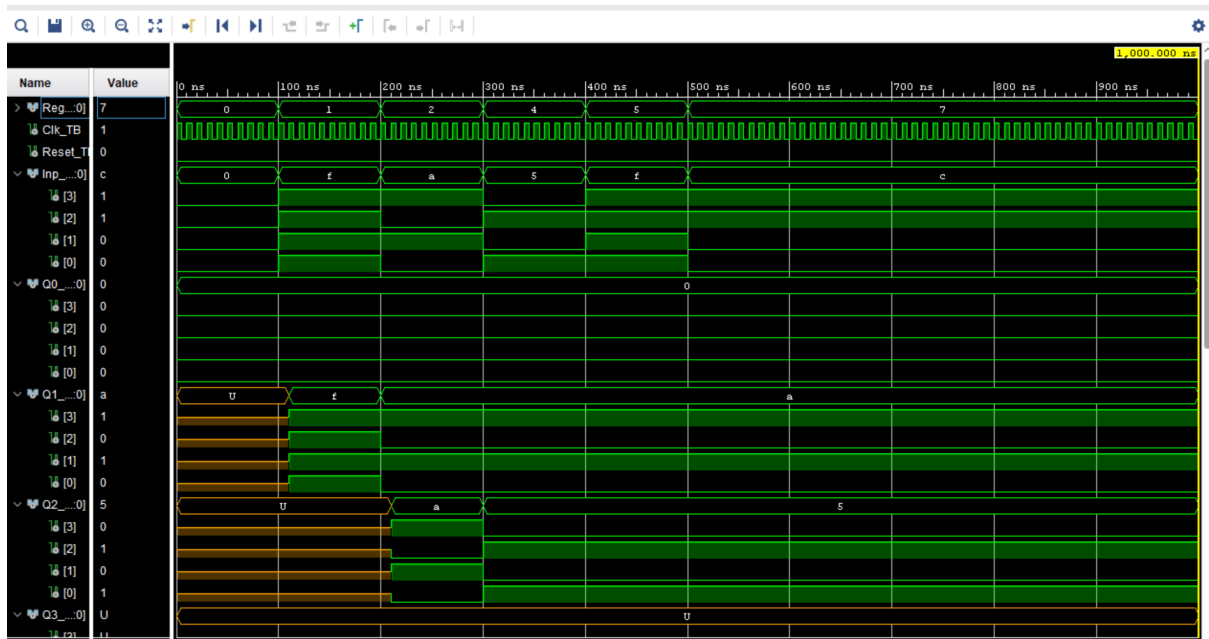
end Behavioral;

```

4.4.3 Elaborated design schematic



4.4.4 Timing Diagram



4.5 Decoder 3 to 8

4.5.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Decoder_3_to_8 is
Port ( I : in STD_LOGIC_VECTOR (2 downto 0);
      EN : in STD_LOGIC;
      Y : out STD_LOGIC_VECTOR (7 downto 0));
end Decoder_3_to_8;

architecture Behavioral of Decoder_3_to_8 is
begin
process (I, EN)
begin
    Y <= (others => '0');
    if EN = '1' then
        Y(to_integer(unsigned(I))) <= '1';
    end if;
end process;
end Behavioral;
```

4.5.2 Simulation File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Decoder_3_to_8 is
end TB_Decoder_3_to_8;

architecture Behavioral of TB_Decoder_3_to_8 is

    component Decoder_3_to_8
        port ( I : in std_logic_vector(2 downto 0);
              EN : in std_logic;
              Y : out std_logic_vector(7 downto 0));
    end component;

    signal I : std_logic_vector(2 downto 0);
    signal EN : std_logic;
    signal Y : std_logic_vector(7 downto 0);

begin

    UUT: Decoder_3_to_8 port map(
        I => I,
        EN => EN,
        Y => Y);

    process
    begin
        EN <= '1';
        I <= "000"; WAIT FOR 50ns;
```

```

I <= "001"; WAIT FOR 50ns;
I <= "010"; WAIT FOR 50ns;
I <= "011"; WAIT FOR 50ns;
I <= "100"; WAIT FOR 50ns;
I <= "101"; WAIT FOR 50ns;
I <= "110"; WAIT FOR 50ns;
I <= "111"; WAIT FOR 50ns;

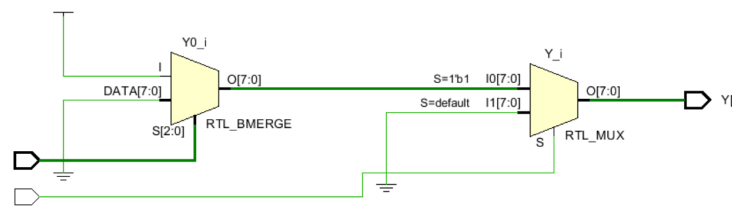
EN <= '0';
I <= "000"; WAIT FOR 50ns;
I <= "001"; WAIT FOR 50ns;
I <= "010"; WAIT FOR 50ns;
I <= "011"; WAIT FOR 50ns;
I <= "100"; WAIT FOR 50ns;
I <= "101"; WAIT FOR 50ns;
I <= "110"; WAIT FOR 50ns;
I <= "111"; WAIT FOR 50ns;

WAIT;
end process;

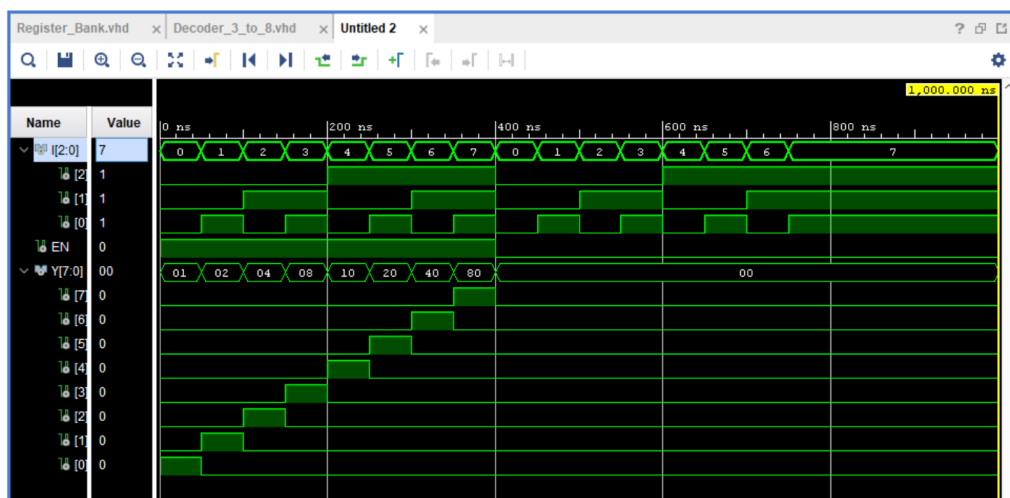
end Behavioral;

```

4.5.3 Elaborated design schematic



4.5.4 Timing Diagram



4.6 Program Counter

4.6.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity Program_Counter is
    Port ( D : in STD_LOGIC_VECTOR (2 downto 0);
          En : in STD_LOGIC;
          Reset : in STD_LOGIC;
          Clk : in STD_LOGIC;
          Q : out STD_LOGIC_VECTOR (2 downto 0));
end Program_Counter;

architecture Behavioral of Program_Counter is
    signal PC : STD_LOGIC_VECTOR(2 downto 0) := (others => '0');

begin

    process (Clk)
    begin
        if rising_edge(Clk) then
            if Reset = '1' then
                PC <= (others => '0');
            elsif En = '1' then
                PC <= D;
            end if;
        end if;
    end process;

    Q <= PC;

end Behavioral;
```

4.6.2 Simulation File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Program_Counter is
-- Port ( );
end TB_Program_Counter;

architecture Behavioral of TB_Program_Counter is

    component Program_Counter is
        Port (
            D      : in  STD_LOGIC_VECTOR (2 downto 0);
            En      : in  STD_LOGIC;
            Reset   : in  STD_LOGIC;
            Clk     : in  STD_LOGIC;
            Q       : out STD_LOGIC_VECTOR (2 downto 0)
        );
    end component;

    signal D_TB      : STD_LOGIC_VECTOR (2 downto 0) := "000";
```

```

signal En_TB      : STD_LOGIC := '0';
signal Reset_TB   : STD_LOGIC := '0';
signal Clk_TB     : STD_LOGIC := '0';
signal Q_TB       : STD_LOGIC_VECTOR (2 downto 0);

```

```
begin
```

```

    UUT : Program_Counter
        port map (
            D      => D_TB,
            En      => En_TB,
            Reset   => Reset_TB,
            Clk     => Clk_TB,
            Q       => Q_TB
        );

```

```

    clk_process : process
begin

```

```

    Clk_TB <= '0';
    wait for 5 ns;
    Clk_TB <= '1';
    wait for 5 ns;

```

```
end process;
```

```

    stim_proc : process
begin

```

```

    Reset_TB <= '1';
    wait for 100 ns;

```

```

    Reset_TB <= '0';
    wait for 100 ns;

```

```

    D_TB  <= "110";
    En_TB <= '1';
    wait for 100 ns;

```

```

    D_TB  <= "001";
    En_TB <= '0';
    wait for 100 ns;

```

```

    En_TB <= '1';
    wait for 100 ns;

```

```

    D_TB  <= "010";
    En_TB <= '1';
    wait for 100 ns;

```

```

    D_TB  <= "011";
    En_TB <= '1';
    wait for 100 ns;
    Reset_TB <= '1';
    wait for 100 ns;

```

```

    Reset_TB <= '0';
    wait for 100 ns;

```

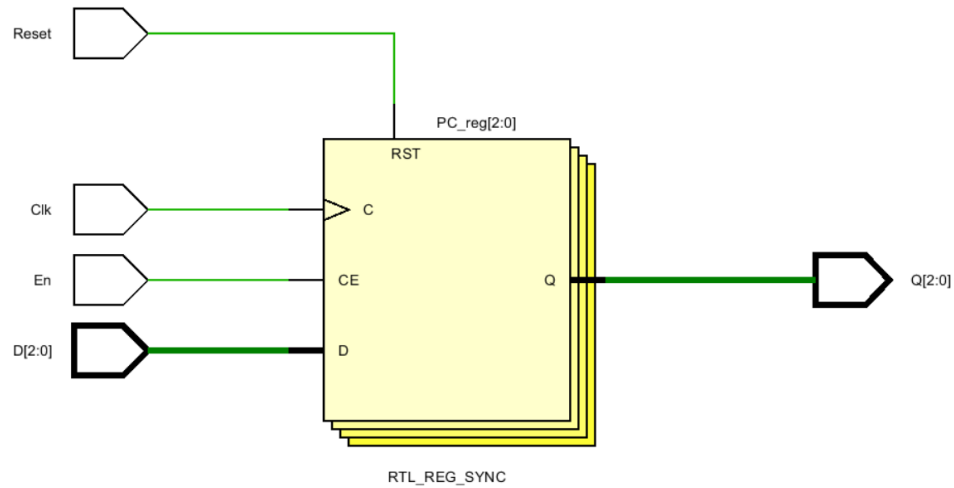
```

    wait;
end process;

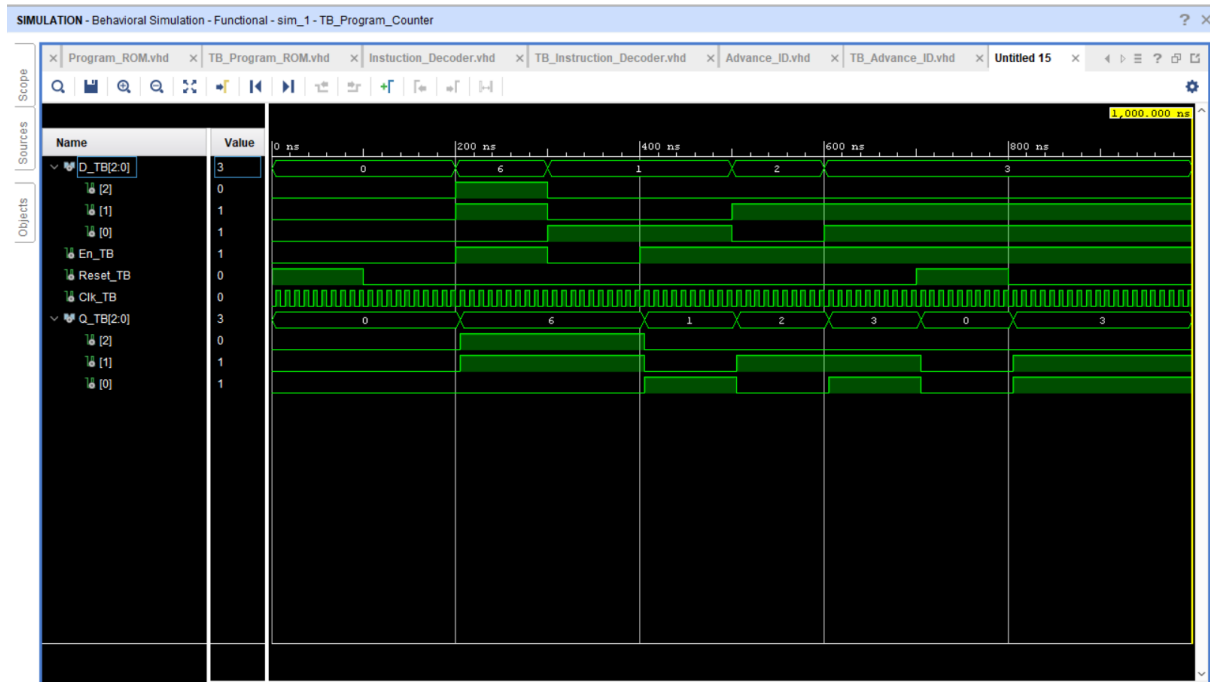
```

```
end Behavioral;
```

4.6.3 Elaborated design schematic



4.6.4 Timing Diagram



4.7 Program ROM

4.7.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Program_ROM is
    Port ( address : in  STD_LOGIC_VECTOR(2 downto 0);
          data      : out STD_LOGIC_VECTOR(11 downto 0));
end Program_ROM;

architecture Behavioral of Program_ROM is

    type rom_type is array (0 to 7) of std_logic_vector(11 downto 0);

    signal ROM_DATA : rom_type := (
        0 => "101110000001",
        1 => "101010000010",
        2 => "101000000011",
        3 => "001111010000",
        4 => "001111000000",
        5 => "000000000000",
        6 => "000000000000",
        7 => "000000000000"
    );

begin

    data <= ROM_DATA(to_integer(unsigned(address)));

end Behavioral;
```

4.7.2 Simulation File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Program_ROM is
-- Port ( );
end TB_Program_ROM;

architecture Behavioral of TB_Program_ROM is

    component Program_ROM
        Port ( address : in  STD_LOGIC_VECTOR (2 downto 0);
              data      : out STD_LOGIC_VECTOR (11 downto 0));
    end component;

    signal address_tb : std_logic_vector(2 downto 0) := "000";
    signal data_tb     : std_logic_vector(11 downto 0);

begin

    uut : Program_ROM
```

```

    port map (
        address => address_tb,
        data     => data_tb
    );

process
begin
    address_tb <= "000";
    wait for 100 ns;

    address_tb <= "001";
    wait for 100 ns;

    address_tb <= "010";
    wait for 100 ns;

    address_tb <= "011";
    wait for 100 ns;

    address_tb <= "100";
    wait for 100 ns;

    address_tb <= "101";
    wait for 100 ns;

    address_tb <= "110";
    wait for 100 ns;

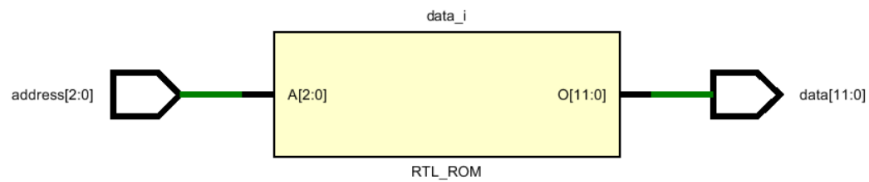
    address_tb <= "111";
    wait for 100 ns;

    wait;
end process;

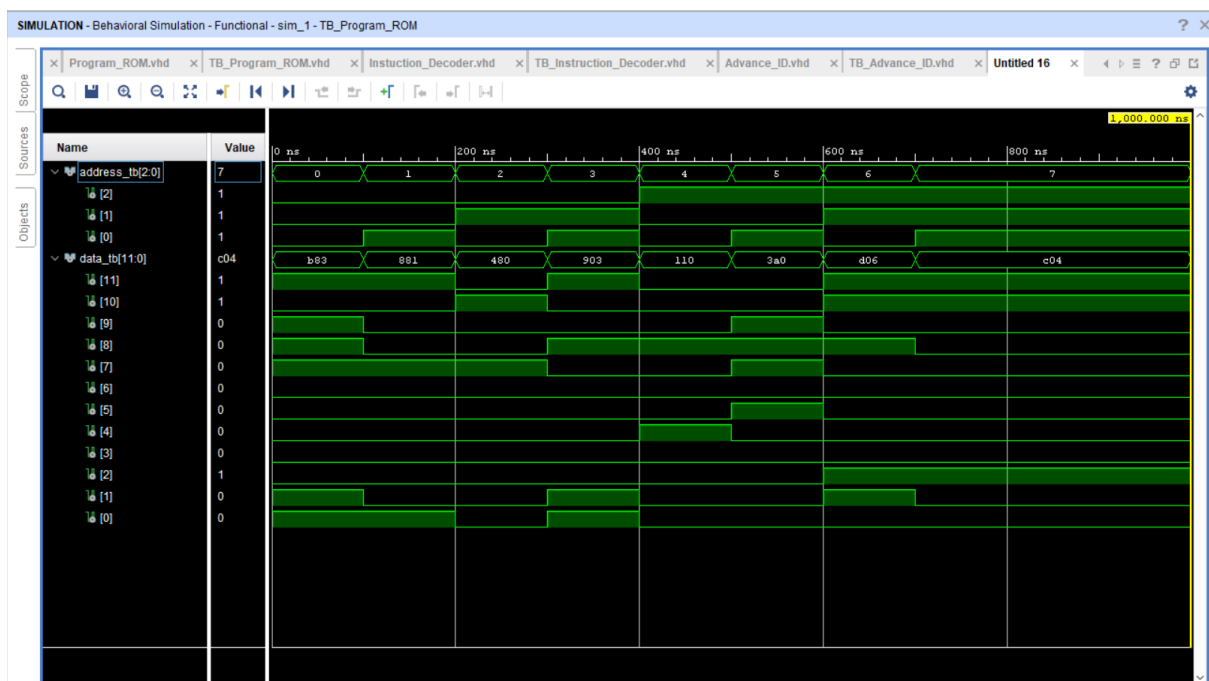
end Behavioral;

```

4.7.3 Elaborated design schematic



4.7.4 Timing Diagram



4.8 Slow Clock

4.8.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

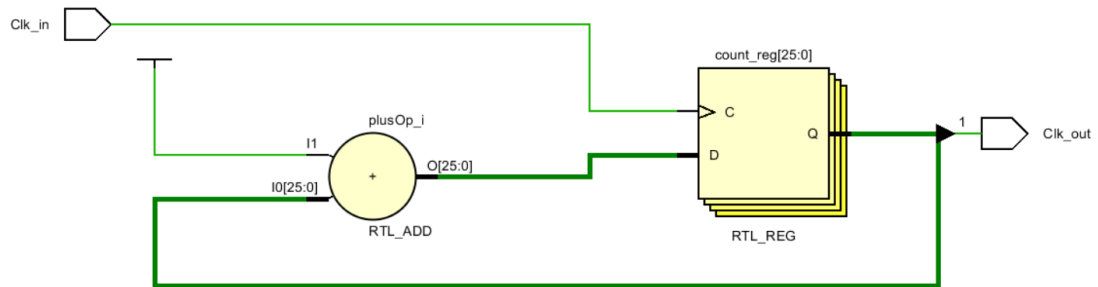
entity Slow_Clk is
Port ( Clk_in : in STD_LOGIC;
       Clk_out : out STD_LOGIC);
end Slow_Clk;

architecture Behavioral of Slow_Clk is
signal count : unsigned(25 downto 0) := (others => '0');

begin
process (Clk_in)
begin
if rising_edge(Clk_in) then
    count <= count + 1;
end if;
end process;

Clk_out <= count(1);
end Behavioral;
```

4.8.2 Elaborated design schematic



4.9 Mux_2_way_3_bit

4.9.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_2_way_3_Bit is
Port (
Adder_out : in STD_LOGIC_VECTOR (2 downto 0);
Address : in STD_LOGIC_VECTOR (2 downto 0);
Jump : in STD_LOGIC;
Q : out STD_LOGIC_VECTOR (2 downto 0)
);
end MUX_2_way_3_Bit;

architecture Behavioral of MUX_2_way_3_Bit is
begin
    process(Adder_out, Address, Jump)
    begin
        case Jump is
            when '0' => Q <= Adder_out;
            when others => Q <= Address;
        end case;
    end process;
end Behavioral;
```

4.9.2 Simulation File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_MUX_2_way_3_Bit is
end TB_MUX_2_way_3_Bit;

architecture Behavioral of TB_MUX_2_way_3_Bit is

    component MUX_2_way_3_Bit
        Port (
            Adder_out : in STD_LOGIC_VECTOR (2 downto 0);
            Address : in STD_LOGIC_VECTOR (2 downto 0);
            Jump : in STD_LOGIC;
            Q : out STD_LOGIC_VECTOR (2 downto 0)
        );
    end component;

    signal Adder_out_TB : STD_LOGIC_VECTOR (2 downto 0) := "000";
    signal Address_TB : STD_LOGIC_VECTOR (2 downto 0) := "000";
    signal Jump_TB : STD_LOGIC := '0';
    signal Q_TB : STD_LOGIC_VECTOR (2 downto 0);

begin

    UUT : MUX_2_way_3_Bit
        port map (
            Adder_out => Adder_out_TB,
```

```

        Address    => Address_TB,
        Jump       => Jump_TB,
        Q          => Q_TB
    );

stimulus : process
begin

    Adder_out_TB <= "001";
    Address_TB   <= "111";
    Jump_TB      <= '0';
    wait for 100 ns;

    Adder_out_TB <= "010";
    Address_TB   <= "110";
    Jump_TB      <= '0';
    wait for 100 ns;

    Adder_out_TB <= "011";
    Address_TB   <= "101";
    Jump_TB      <= '0';
    wait for 100 ns;

    Adder_out_TB <= "100";
    Address_TB   <= "000";
    Jump_TB      <= '0';
    wait for 100 ns;

    Adder_out_TB <= "111";
    Address_TB   <= "010";
    Jump_TB      <= '0';
    wait for 100 ns;

    Adder_out_TB <= "101";
    Address_TB   <= "000";
    Jump_TB      <= '1';
    wait for 100 ns;

    Adder_out_TB <= "001";
    Address_TB   <= "011";
    Jump_TB      <= '1';
    wait for 100 ns;

    Adder_out_TB <= "010";
    Address_TB   <= "111";
    Jump_TB      <= '1';
    wait for 100 ns;

    Adder_out_TB <= "100";
    Address_TB   <= "101";
    Jump_TB      <= '1';
    wait for 100 ns;

    Adder_out_TB <= "011";
    Address_TB   <= "110";
    Jump_TB      <= '1';
    wait for 50 ns;

```

```

    Jump_TB <= '0';
    wait for 50 ns;

    Adder_out_TB <= "010";
    Address_TB    <= "101";
    Jump_TB      <= '0';
    wait for 50 ns;

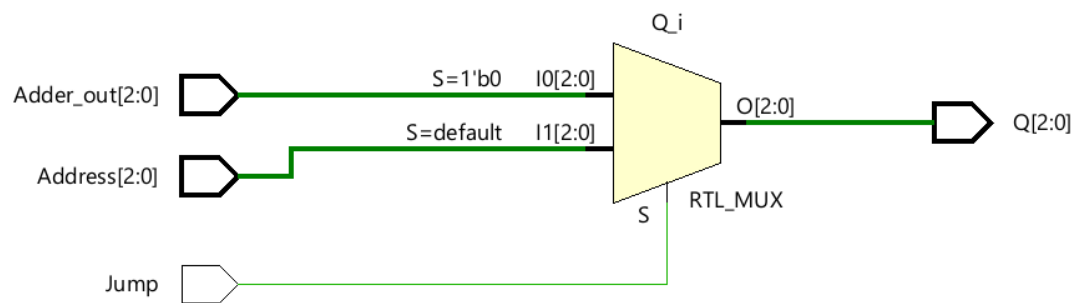
    Jump_TB <= '1';
    wait for 50 ns;

    wait;
end process;

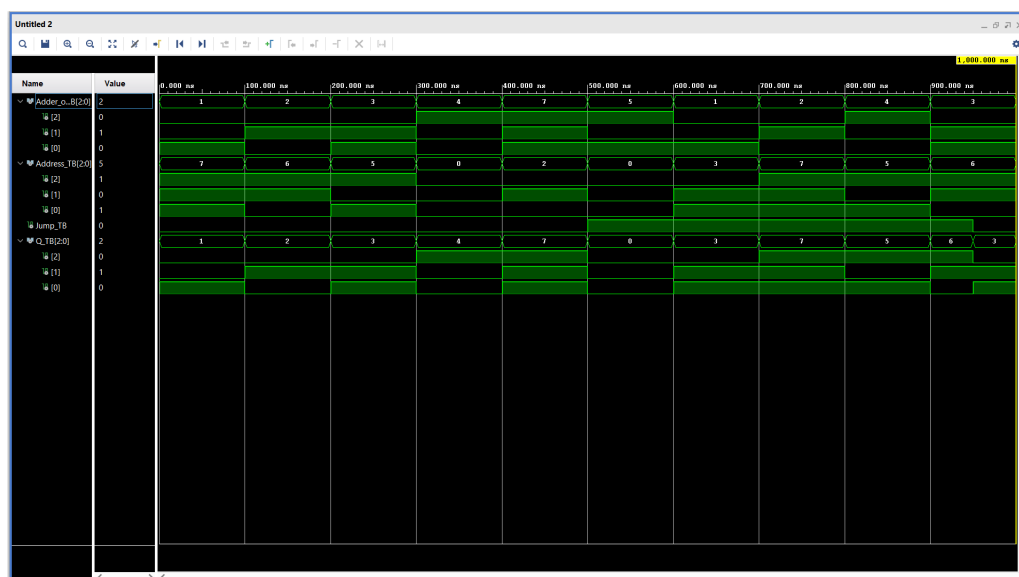
end Behavioral;

```

4.9.3 Elaborated design schematic



4.9.4 Timing Diagram



4.10 Mux_2_way_4_bit

4.10.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_2_way_4_Bit is
    Port (
        Adder_Sub_Out : in  STD_LOGIC_VECTOR (3 downto 0);
        Imd_Value      : in  STD_LOGIC_VECTOR (3 downto 0);
        Load          : in  STD_LOGIC;
        Q              : out STD_LOGIC_VECTOR (3 downto 0)
    );
end MUX_2_way_4_Bit;

architecture Behavioral of MUX_2_way_4_Bit is
begin

    with Load select
        Q <= Adder_Sub_Out when '0',
            Imd_Value      when others;

end Behavioral;
```

4.10.2 Simulation File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_MUX_2_way_4_Bit is
end TB_MUX_2_way_4_Bit;

architecture Behavioral of TB_MUX_2_way_4_Bit is

    component MUX_2_way_4_Bit
        Port (
            Adder_Sub_Out : in  STD_LOGIC_VECTOR (3 downto 0);
            Imd_Value      : in  STD_LOGIC_VECTOR (3 downto 0);
            Load          : in  STD_LOGIC;
            Q              : out STD_LOGIC_VECTOR (3 downto 0)
        );
    end component;

    signal Adder_Sub_Out_TB : STD_LOGIC_VECTOR (3 downto 0) := "0000";
    signal Imd_Value_TB    : STD_LOGIC_VECTOR (3 downto 0) := "0000";
    signal Load_TB        : STD_LOGIC := '0';
    signal Q_TB            : STD_LOGIC_VECTOR (3 downto 0);

begin

    UUT : MUX_2_way_4_Bit
        port map (
            Adder_Sub_Out => Adder_Sub_Out_TB,
            Imd_Value     => Imd_Value_TB,
            Load          => Load_TB,
```

```

        Q          => Q_TB
    );

stimulus : process
begin

    Adder_Sub_Out_TB <= "0000";
    Imd_Value_TB     <= "1111";
    Load_TB        <= '0';
    wait for 100 ns;

    Adder_Sub_Out_TB <= "1010";
    Imd_Value_TB     <= "0101";
    Load_TB        <= '0';
    wait for 100 ns;

    Adder_Sub_Out_TB <= "1111";
    Imd_Value_TB     <= "0000";
    Load_TB        <= '0';
    wait for 100 ns;

    Adder_Sub_Out_TB <= "0110";
    Imd_Value_TB     <= "1001";
    Load_TB        <= '0';
    wait for 100 ns;

    Adder_Sub_Out_TB <= "0000";
    Imd_Value_TB     <= "0001";
    Load_TB        <= '1';
    wait for 100 ns;

    Adder_Sub_Out_TB <= "1010";
    Imd_Value_TB     <= "0010";
    Load_TB        <= '1';
    wait for 100 ns;

    Adder_Sub_Out_TB <= "1111";
    Imd_Value_TB     <= "0011";
    Load_TB        <= '1';
    wait for 100 ns;

    Adder_Sub_Out_TB <= "0101";
    Imd_Value_TB     <= "1111";
    Load_TB        <= '1';
    wait for 100 ns;

    Adder_Sub_Out_TB <= "1100";
    Imd_Value_TB     <= "0011";
    Load_TB        <= '1';
    wait for 50 ns;

    Load_TB <= '0';
    wait for 50 ns;

    Adder_Sub_Out_TB <= "1001";
    Imd_Value_TB     <= "0110";
    Load_TB        <= '0';
    wait for 50 ns;

```

```

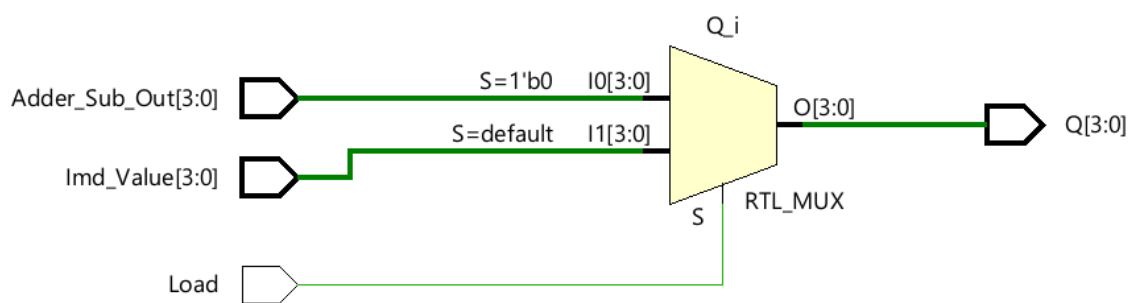
        Load_TB <= '1';
        wait for 50 ns;

        wait;
    end process;

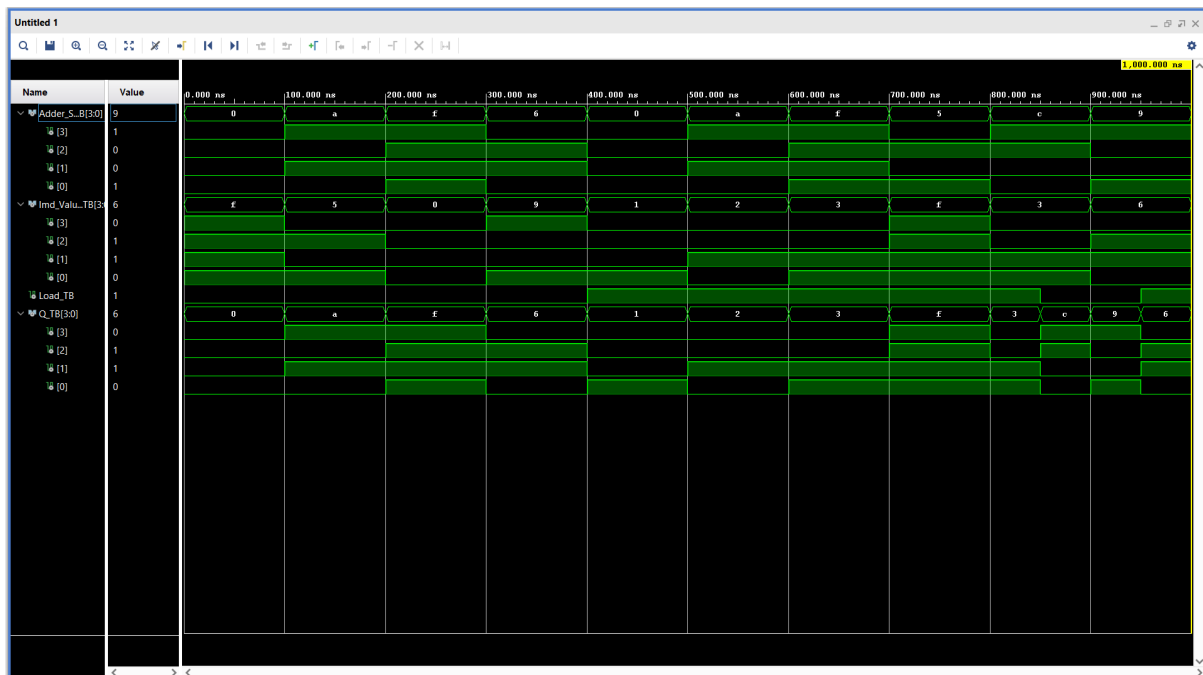
end Behavioral;

```

4.10.3 Elaborated design schematic



4.10.4 Timing Diagram



4.11 Mux_8_way_4_bit

4.11.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_8_way_4_Bit is
    Port (
        Reg_Sel : in  STD_LOGIC_VECTOR (2 downto 0);
        R0      : in  STD_LOGIC_VECTOR (3 downto 0);
        R1      : in  STD_LOGIC_VECTOR (3 downto 0);
        R2      : in  STD_LOGIC_VECTOR (3 downto 0);
        R3      : in  STD_LOGIC_VECTOR (3 downto 0);
        R4      : in  STD_LOGIC_VECTOR (3 downto 0);
        R5      : in  STD_LOGIC_VECTOR (3 downto 0);
        R6      : in  STD_LOGIC_VECTOR (3 downto 0);
        R7      : in  STD_LOGIC_VECTOR (3 downto 0);
        Q       : out STD_LOGIC_VECTOR (3 downto 0)
    );
end MUX_8_way_4_Bit;

architecture Behavioral of MUX_8_way_4_Bit is
begin

    process(Reg_Sel, R0, R1, R2, R3, R4, R5, R6, R7)
    begin
        case Reg_Sel is
            when "000" => Q <= R0;
            when "001" => Q <= R1;
            when "010" => Q <= R2;
            when "011" => Q <= R3;
            when "100" => Q <= R4;
            when "101" => Q <= R5;
            when "110" => Q <= R6;
            when others => Q <= R7;
        end case;
    end process;
end Behavioral;
```

4.11.2 Simulation File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_MUX_8_way_4_Bit is
end TB_MUX_8_way_4_Bit;

architecture Behavioral of TB_MUX_8_way_4_Bit is

    component MUX_8_way_4_Bit
        Port (
            Reg_Sel : in  STD_LOGIC_VECTOR (2 downto 0);
            R0      : in  STD_LOGIC_VECTOR (3 downto 0);
            R1      : in  STD_LOGIC_VECTOR (3 downto 0);
```

```

        R2      : in  STD_LOGIC_VECTOR (3 downto 0);
        R3      : in  STD_LOGIC_VECTOR (3 downto 0);
        R4      : in  STD_LOGIC_VECTOR (3 downto 0);
        R5      : in  STD_LOGIC_VECTOR (3 downto 0);
        R6      : in  STD_LOGIC_VECTOR (3 downto 0);
        R7      : in  STD_LOGIC_VECTOR (3 downto 0);
        Q       : out STD_LOGIC_VECTOR (3 downto 0)
    );
end component;

signal Reg_Sel_TB : STD_LOGIC_VECTOR (2 downto 0) := "000";

signal R0_TB : STD_LOGIC_VECTOR (3 downto 0) := "0000";
signal R1_TB : STD_LOGIC_VECTOR (3 downto 0) := "0001";
signal R2_TB : STD_LOGIC_VECTOR (3 downto 0) := "0010";
signal R3_TB : STD_LOGIC_VECTOR (3 downto 0) := "0011";
signal R4_TB : STD_LOGIC_VECTOR (3 downto 0) := "0100";
signal R5_TB : STD_LOGIC_VECTOR (3 downto 0) := "0101";
signal R6_TB : STD_LOGIC_VECTOR (3 downto 0) := "0110";
signal R7_TB : STD_LOGIC_VECTOR (3 downto 0) := "0111";

signal Q_TB : STD_LOGIC_VECTOR (3 downto 0);

begin

    UUT : MUX_8_way_4_Bit
        port map (
            Reg_Sel => Reg_Sel_TB,
            R0 => R0_TB, R1 => R1_TB, R2 => R2_TB, R3 => R3_TB,
            R4 => R4_TB, R5 => R5_TB, R6 => R6_TB, R7 => R7_TB,
            Q  => Q_TB
        );

    stimulus : process
    begin

        Reg_Sel_TB <= "000"; wait for 100 ns;

        Reg_Sel_TB <= "001"; wait for 100 ns;

        Reg_Sel_TB <= "010"; wait for 100 ns;

        Reg_Sel_TB <= "011"; wait for 100 ns;

        Reg_Sel_TB <= "100"; wait for 100 ns;

        Reg_Sel_TB <= "101"; wait for 100 ns;

        Reg_Sel_TB <= "110"; wait for 100 ns;

        Reg_Sel_TB <= "111"; wait for 100 ns;

        R0_TB <= "0000";
        R1_TB <= "0000";
        R2_TB <= "0000";
        R3_TB <= "0000";
        R4_TB <= "0011";
        R5_TB <= "0010";

```

```

R6_TB <= "0000";
R7_TB <= "0001";
wait for 10 ns;

Reg_Sel_TB <= "111"; wait for 100 ns;

Reg_Sel_TB <= "101"; wait for 100 ns;

R7_TB <= "0011"; wait for 10 ns;

Reg_Sel_TB <= "111"; wait for 100 ns;

Reg_Sel_TB <= "100"; wait for 100 ns;

R7_TB <= "0110"; wait for 10 ns;

Reg_Sel_TB <= "111"; wait for 100 ns;

R0_TB <= "0000";
Reg_Sel_TB <= "000"; wait for 100 ns;

R7_TB      <= "1010";
Reg_Sel_TB <= "111";
wait for 50 ns;

R7_TB <= "0101";
wait for 50 ns;

R4_TB      <= "1100";
R5_TB      <= "0011";
Reg_Sel_TB <= "100";
wait for 50 ns;

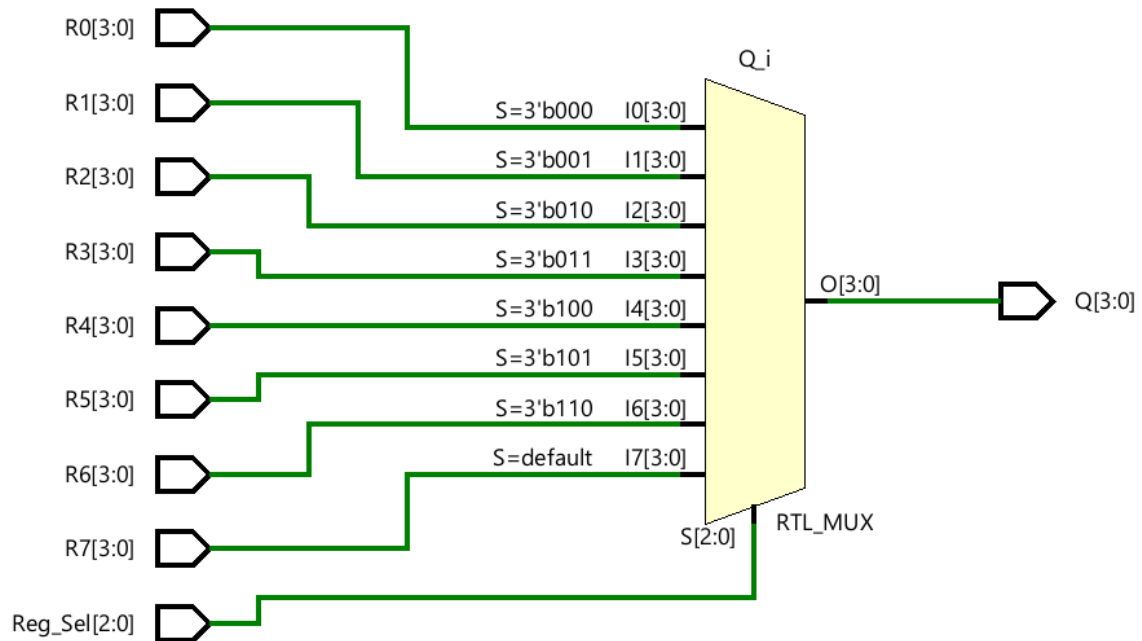
Reg_Sel_TB <= "101";
wait for 50 ns;

wait;
end process;

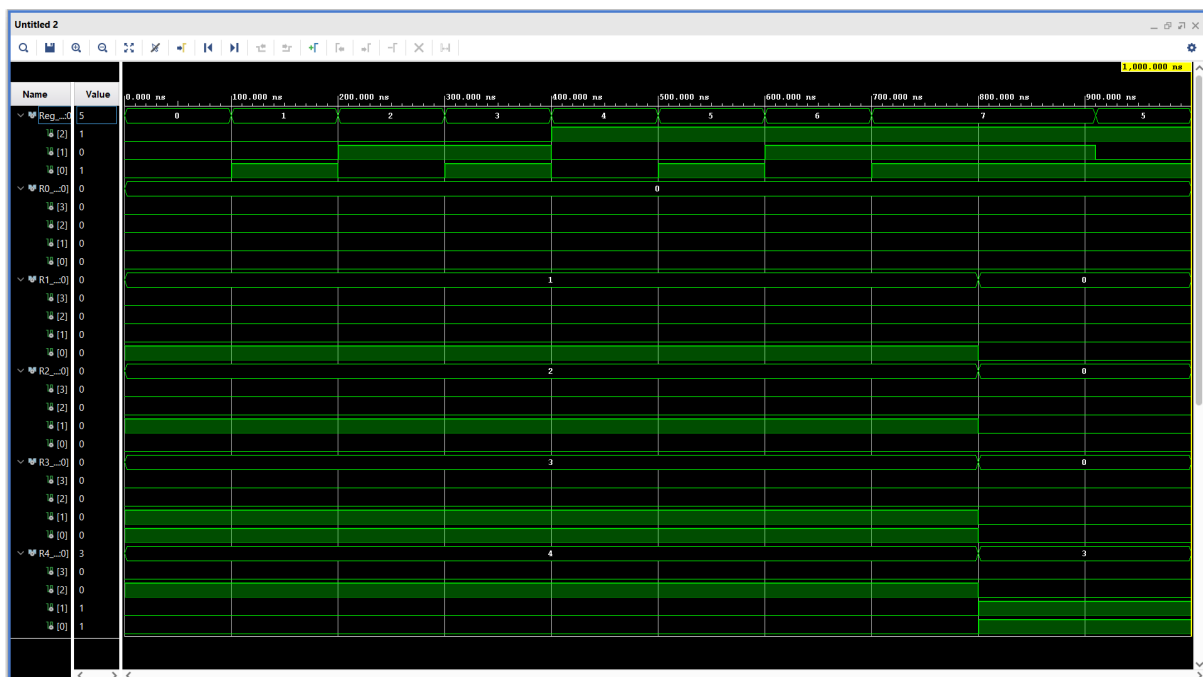
end Behavioral;

```

4.11.3 Elaborated design schematic



4.11.4 Timing Diagram





4.12 LUT_16_7

4.12.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity LUT_16_7 is
PORT (
    Data : IN STD_LOGIC_VECTOR (3 DOWNTO 0);
    Seg7_Out : OUT STD_LOGIC_VECTOR (6 DOWNTO 0);
    Anode : OUT STD_LOGIC_VECTOR (3 DOWNTO 0)
);
END LUT_16_7;

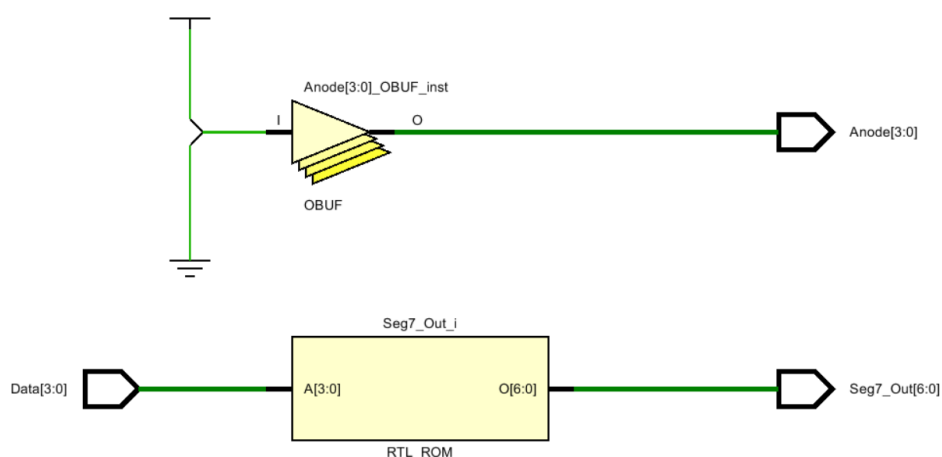
architecture Behavioral of LUT_16_7 is

    type rom_type is array (0 to 15) of STD_LOGIC_VECTOR(6 downto 0);

    signal LUT_16_7_ROM : rom_type := (
        "1000000", "1111001", "0100100", "0110000",
        "0011001", "0010010", "0000010", "1111000",
        "0000000", "0010000", "0001000", "0000011",
        "1000110", "0100001", "0000110", "0001110"
    );

begin
    Seg7_Out <= LUT_16_7_ROM(to_integer(unsigned(Data)));
    Anode <= "1110";
end Behavioral;
```

4.12.2 Elaborated design schematic



4.13 Instruction Decoder

4.13.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Instruction_Decoder is
    Port (
        Instruction      : in  STD_LOGIC_VECTOR (11 downto 0);
        Reg_Check_Jmp    : in  STD_LOGIC_VECTOR (3 downto 0);
        Reg_en           : out STD_LOGIC_VECTOR (2 downto 0);
        Load_sel         : out STD_LOGIC;
        Imd_Val          : out STD_LOGIC_VECTOR (3 downto 0);
        Reg_Sel_A         : out STD_LOGIC_VECTOR (2 downto 0);
        Reg_Sel_B         : out STD_LOGIC_VECTOR (2 downto 0);
        Add_Sub_Sel      : out STD_LOGIC;
        Jump_address     : out STD_LOGIC_VECTOR (2 downto 0);
        Jump_flag        : out STD_LOGIC
    );
end Instruction_Decoder;

architecture Behavioral of Instruction_Decoder is
begin
    process(Instruction, Reg_Check_Jmp)
    begin
        Reg_en      <= "000";
        Load_sel    <= '0';
        Imd_Val     <= "0000";
        Reg_Sel_A   <= "000";
        Reg_Sel_B   <= "000";
        Add_Sub_Sel <= '0';
        Jump_address <= "000";
        Jump_flag   <= '0';

        case Instruction(11 downto 10) is

            when "00" =>
                Reg_en      <= Instruction(9 downto 7);
                Reg_Sel_A   <= Instruction(9 downto 7);
                Reg_Sel_B   <= Instruction(6 downto 4);

            when "01" =>
                Reg_en      <= Instruction(9 downto 7);
                Reg_Sel_A   <= "000";
                Reg_Sel_B   <= Instruction(9 downto 7);
                Add_Sub_Sel <= '1';

            when "10" =>
                Reg_en      <= Instruction(9 downto 7);
                Imd_Val     <= Instruction(3 downto 0);
                Load_sel    <= '1';

            when "11" =>
                Reg_Sel_B   <= Instruction(9 downto 7);
                Jump_address <= Instruction(2 downto 0);
                if (Reg_Check_Jmp = "0000") then
                    Jump_flag <= '1';
                end if;
            end case;
        end process;
    end begin;
end Behavioral;
```

```

        else
            Jump_flag <= '0';
        end if;

        when others =>
            null;

    end case;
end process;
end Behavioral;

```

4.13.2 Simulation File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Instruction_Decoder is
end TB_Instruction_Decoder;

architecture Behavioral of TB_Instruction_Decoder is

    component Instruction_Decoder
        Port (
            Instruction      : in  STD_LOGIC_VECTOR (11 downto 0);
            Reg_Check_Jmp    : in  STD_LOGIC_VECTOR (3 downto 0);
            Reg_en           : out STD_LOGIC_VECTOR (2 downto 0);
            Load_sel         : out STD_LOGIC;
            Imd_Val          : out STD_LOGIC_VECTOR (3 downto 0);
            Reg_Sel_A        : out STD_LOGIC_VECTOR (2 downto 0);
            Reg_Sel_B        : out STD_LOGIC_VECTOR (2 downto 0);
            Add_Sub_Sel      : out STD_LOGIC;
            Jump_address     : out STD_LOGIC_VECTOR (2 downto 0);
            Jump_flag        : out STD_LOGIC
        );
    end component;

    signal Instruction_TB    : STD_LOGIC_VECTOR (11 downto 0) := (others => '0');
    signal Reg_Check_Jmp_TB : STD_LOGIC_VECTOR (3 downto 0)  := (others => '0');

    signal Reg_en_TB        : STD_LOGIC_VECTOR (2 downto 0)  := (others => '0');
    signal Load_sel_TB     : STD_LOGIC                      := '0';
    signal Imd_Val_TB       : STD_LOGIC_VECTOR (3 downto 0)  := (others => '0');
    signal Reg_SelA_TB      : STD_LOGIC_VECTOR (2 downto 0)  := (others => '0');
    signal Reg_SelB_TB      : STD_LOGIC_VECTOR (2 downto 0)  := (others => '0');
    signal Add_SubSel_TB    : STD_LOGIC                      := '0';
    signal Jump_add_TB      : STD_LOGIC_VECTOR (2 downto 0)  := (others => '0');
    signal Jump_flag_TB     : STD_LOGIC                      := '0';

begin

```

```

    uut: Instruction_Decoder port map (
        Instruction    => Instruction_TB,
        Reg_Check_Jmp => Reg_Check_Jmp_TB,
        Reg_en        => Reg_en_TB,
        Load_sel      => Load_sel_TB,
        Imd_Val        => Imd_Val_TB,
        Reg_Sel_A      => Reg_SelA_TB,
        Reg_Sel_B      => Reg_SelB_TB,
        Add_Sub_Sel    => Add_SubSel_TB,
        Jump_address   => Jump_add_TB,
        Jump_flag      => Jump_flag_TB
    );

    stim_proc: process
    begin
        Instruction_TB <= "101110000000";
        Reg_Check_Jmp_TB <= "0000";
        wait for 100 ns;

        Instruction_TB <= "100010000011";
        wait for 100 ns;

        Instruction_TB <= "010100000000";
        wait for 100 ns;

        Instruction_TB <= "001110010000";
        wait for 100 ns;

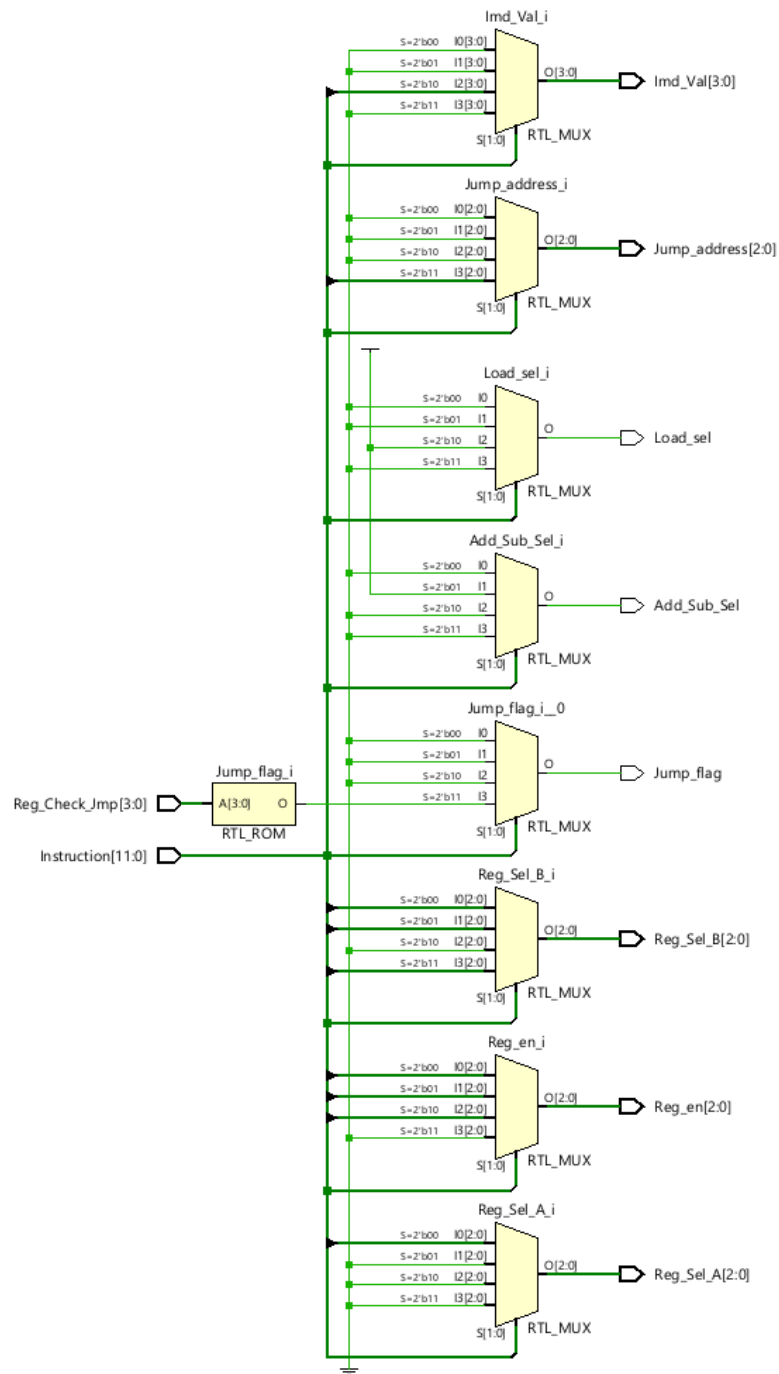
        Instruction_TB <= "110010000111";
        Reg_Check_Jmp_TB <= "0000";
        wait for 100 ns;

        Reg_Check_Jmp_TB <= "0101";
        wait for 100 ns;

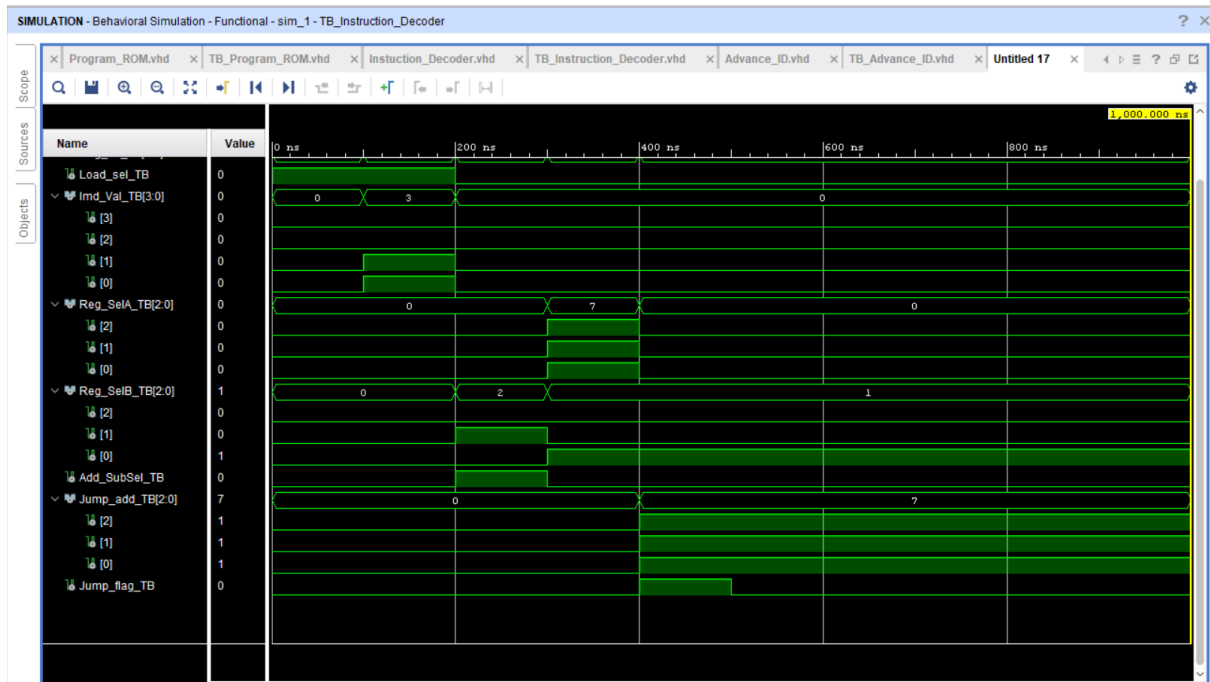
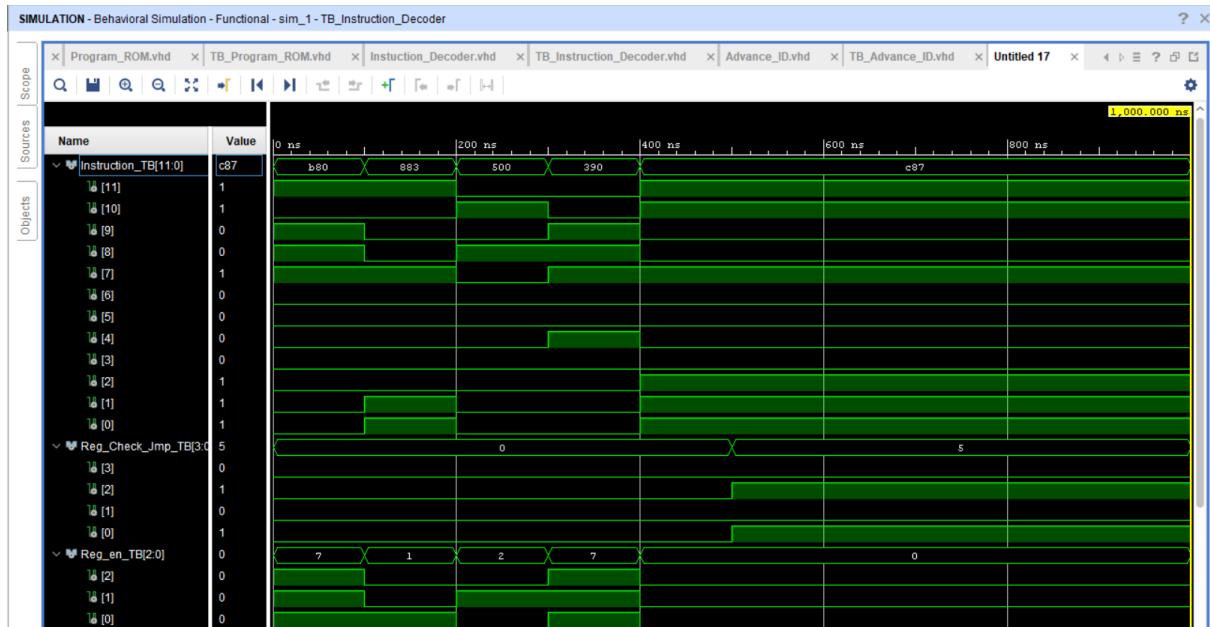
        wait;
    end process;
end Behavioral;

```

4.13.3 Elaborated design schematic



4.13.4 Timing Diagram



4.14 ALU

4.14.1 Design File

Listing 1: 2-bit Multiplier

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Add_Sub_Unit is
Port (
    A : in STD_LOGIC_VECTOR (3 downto 0);
    B : in STD_LOGIC_VECTOR (3 downto 0);
    Sel : in STD_LOGIC; -- 0 = ADD, 1 = SUB
    En : in STD_LOGIC;
    S : out STD_LOGIC_VECTOR (3 downto 0);
    C_out : out STD_LOGIC;
    Z_out : out STD_LOGIC
);
end Add_Sub_Unit;

architecture Behavioral of Add_Sub_Unit is
begin
process(A, B, Sel, En)
    variable temp_res : unsigned(4 downto 0);
begin
    if En = '1' then
        if Sel = '0' then
            temp_res := ('0' & unsigned(A)) + ('0' & unsigned(B));
        else
            temp_res := ('0' & unsigned(A)) - ('0' & unsigned(B));
        end if;

        S      <= std_logic_vector(temp_res(3 downto 0));
        C_out <= temp_res(4);

        if temp_res(3 downto 0) = "0000" then
            Z_out <= '1';
        else
            Z_out <= '0';
        end if;
    else
        S      <= (others => '0');
        C_out <= '0';
        Z_out <= '0';
    end if;
end process;
end Behavioral;
```

4.14.2 Simulation File

Listing 2: 2-bit Multiplier

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```

entity TB_Add_Sub_Unit is
end TB_Add_Sub_Unit;

architecture Behavioral of TB_Add_Sub_Unit is

    component Add_Sub_Unit
        Port (
            A      : in  STD_LOGIC_VECTOR(3 downto 0);
            B      : in  STD_LOGIC_VECTOR(3 downto 0);
            Sel     : in  STD_LOGIC;
            En      : in  STD_LOGIC;
            S       : out STD_LOGIC_VECTOR(3 downto 0);
            C_out   : out STD_LOGIC;
            Z_out   : out STD_LOGIC
        );
    end component;

    signal A_TB, B_TB, S_TB : STD_LOGIC_VECTOR(3 downto 0) := "0000";
    signal Sel_TB, En_TB, C_out_TB, Z_out_TB : STD_LOGIC := '0';

begin

    UUT : Add_Sub_Unit
        port map (
            A      => A_TB,
            B      => B_TB,
            Sel     => Sel_TB,
            En      => En_TB,
            S       => S_TB,
            C_out   => C_out_TB,
            Z_out   => Z_out_TB
        );

    process
    begin

        En_TB <= '0';
        A_TB <= "0101"; B_TB <= "0011"; Sel_TB <= '0'; wait for 100 ns;

        En_TB <= '1'; Sel_TB <= '0';

        A_TB <= "0000"; B_TB <= "0000"; wait for 100 ns;

        A_TB <= "0001"; B_TB <= "0010"; wait for 100 ns;

        A_TB <= "0011"; B_TB <= "0011"; wait for 100 ns;

        A_TB <= "0111"; B_TB <= "0001"; wait for 100 ns;

        A_TB <= "1111"; B_TB <= "0001"; wait for 100 ns;

        A_TB <= "1001"; B_TB <= "1001"; wait for 100 ns;

        En_TB <= '1'; Sel_TB <= '1';

        A_TB <= "0000"; B_TB <= "0000"; wait for 100 ns;

        A_TB <= "0011"; B_TB <= "0001"; wait for 100 ns;
    end process;
end

```

```

A_TB <= "0110"; B_TB <= "0011"; wait for 100 ns;

A_TB <= "0101"; B_TB <= "0101"; wait for 100 ns;

A_TB <= "0000"; B_TB <= "0011"; wait for 100 ns;

A_TB <= "0000"; B_TB <= "0001"; wait for 100 ns;

En_TB <= '1'; Sel_TB <= '0';
A_TB <= "0100"; B_TB <= "0010"; wait for 50 ns;

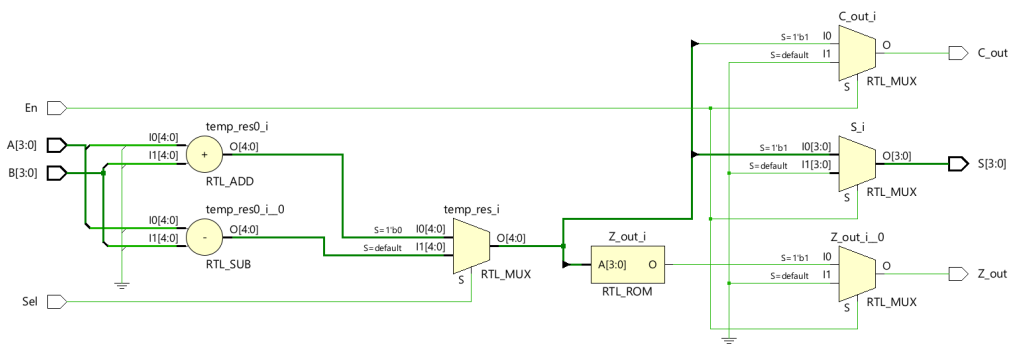
En_TB <= '0'; wait for 50 ns;

wait;
end process;

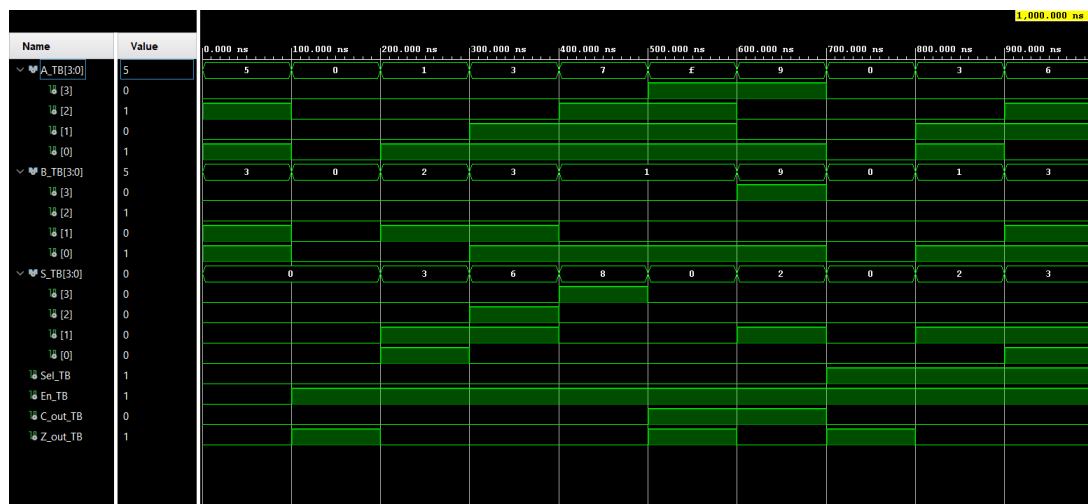
end Behavioral;

```

4.14.3 Elaborated design schematic



4.14.4 Timing Diagram



4.15 Basic Nanoprocessor

4.15.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Nanoprocessor_Basic is
Port (
    Clk : in STD_LOGIC;
    Reset : in STD_LOGIC;
    Overflow : out STD_LOGIC;
    Zero : out STD_LOGIC;
    R7 : out STD_LOGIC_VECTOR (3 downto 0);
    Seg7_out : out STD_LOGIC_VECTOR (6 downto 0);
    Anode : out STD_LOGIC_VECTOR (3 downto 0)
);
end Nanoprocessor_Basic;

architecture Behavioral of Nanoprocessor_Basic is

component LUT_16_7
port(
    Data : IN STD_LOGIC_VECTOR (3 DOWNT0 0);
    Seg7_out : OUT STD_LOGIC_VECTOR (6 DOWNT0 0);
    Anode : OUT STD_LOGIC_VECTOR (3 DOWNT0 0)
);
end component;

component Slow_Clk
Port (
    Clk_in : in STD_LOGIC;
    Clk_out : out STD_LOGIC
);
end component;

component Register_Bank
Port(
    Reg_en : in STD_LOGIC_VECTOR (2 downto 0);
    Clk : in STD_LOGIC;
    Reset : in STD_LOGIC;
    Inp : in STD_LOGIC_VECTOR (3 downto 0);
    Q0,Q1,Q2,Q3,Q4,Q5,Q6,Q7 : out STD_LOGIC_VECTOR (3 downto 0)
);
end component;

component MUX_8_way_4_Bit
Port(
    Reg_Sel : in STD_LOGIC_VECTOR (2 downto 0);
    R0,R1,R2,R3,R4,R5,R6,R7 : in STD_LOGIC_VECTOR (3 downto 0);
    Q : out STD_LOGIC_VECTOR (3 downto 0)
);
end component;

component Add_Sub_Unit
Port(
    A : in STD_LOGIC_VECTOR (3 downto 0);
    B : in STD_LOGIC_VECTOR (3 downto 0);
```

```

        Sel : in STD_LOGIC;
        En : in STD_LOGIC;
        S : out STD_LOGIC_VECTOR (3 downto 0);
        C_out : out STD_LOGIC;
        Z_out : out STD_LOGIC
    );
end component;

component MUX_2_way_4_Bit
Port (
    Adder_Sub_Out : in STD_LOGIC_VECTOR (3 downto 0);
    Imd_Value : in STD_LOGIC_VECTOR (3 downto 0);
    Load : in STD_LOGIC;
    Q : out STD_LOGIC_VECTOR (3 downto 0)
);
end component;

component MUX_2_way_3_Bit
Port (
    Adder_out : in STD_LOGIC_VECTOR (2 downto 0);
    Address : in STD_LOGIC_VECTOR (2 downto 0);
    Jump : in STD_LOGIC;
    Q : out STD_LOGIC_VECTOR (2 downto 0)
);
end component;

component Instruction_Decoder
Port (
    Instruction : in STD_LOGIC_VECTOR (11 downto 0);
    Reg_Check_Jmp : in STD_LOGIC_VECTOR (3 downto 0);
    Reg_en : out STD_LOGIC_VECTOR (2 downto 0);
    Load_sel : out STD_LOGIC;
    Imd_Val : out STD_LOGIC_VECTOR (3 downto 0);
    Reg_Sel_A : out STD_LOGIC_VECTOR (2 downto 0);
    Reg_Sel_B : out STD_LOGIC_VECTOR (2 downto 0);
    Add_Sub_Sel : out STD_LOGIC;
    Jump_address : out STD_LOGIC_VECTOR (2 downto 0);
    Jump_flag : out STD_LOGIC
);
end component;

component Program_ROM
Port (
    address : in STD_LOGIC_VECTOR (2 downto 0);
    data : out STD_LOGIC_VECTOR (11 downto 0)
);
end component;

component Program_Counter
Port (
    D : in STD_LOGIC_VECTOR (2 downto 0);
    En : in STD_LOGIC;
    Reset : in STD_LOGIC;
    Clk : in STD_LOGIC;
    Q : out STD_LOGIC_VECTOR (2 downto 0)
);
end component;

```

```

component Adder_3_Bit
Port (
    A : in STD_LOGIC_VECTOR (2 downto 0);
    C_in : in STD_LOGIC;
    S : out STD_LOGIC_VECTOR (2 downto 0);
    C_out : out STD_LOGIC
);
end component;

signal Imd_Val, Add_Sub_Out, Data_Mux_to_Reg_Bank : std_logic_vector(3
    downto 0);
signal Data_bus0, Data_bus1, Data_bus2, Data_bus3,
    Data_bus4, Data_bus5, Data_bus6, Data_bus7 : std_logic_vector(3
    downto 0);

signal Ins_Bus : STD_LOGIC_VECTOR(11 downto 0);

signal Mem_Bus, Reg_SelA, Reg_SelB, Count_Mux_to_PC,
    Mem_RCA3_to_Mux, Jump_Add : STD_LOGIC_VECTOR (2 downto 0);

signal Reg_bus : STD_LOGIC_VECTOR(2 downto 0);

signal Add_Sub_Sel, Load_Select, Jump_Flag : STD_LOGIC;

signal slow_clock : STD_LOGIC;

signal Mux_A_out, Mux_B_out : std_logic_vector(3 downto 0);

signal Carry_Flag, Zero_Flag : std_logic;

begin

LUT_16_7_0 : LUT_16_7
port map(
    Data => Data_bus7,
    Seg7_out => Seg7_out,
    Anode => Anode
);

Slow_Clk_0 : Slow_Clk
port map (
    Clk_in => Clk,
    Clk_out => slow_clock
);

Program_ROM_0 : Program_ROM
port map (
    address => Mem_Bus,
    data => Ins_Bus
);

Program_Counter_0 : Program_Counter
port map (
    D => Count_Mux_to_PC,
    En => '1',

```

```

        Reset => Reset,
        Clk => slow_clock,
        Q => Mem_Bus
    );

Instruction_Decoder_0 : Instruction_Decoder
port map (
    Instruction => Ins_Bus,
    Reg_en => Reg_bus,
    Load_sel => Load_Select,
    Imd_Val => Imd_Val,
    Reg_Sel_A => Reg_SelA,
    Reg_Sel_B => Reg_SelB,
    Add_Sub_Sel => Add_Sub_Sel,
    Jump_address => Jump_Add,
    Jump_flag => Jump_Flag,
    Reg_Check_Jmp => Mux_A_out
);

Register_Bank_0 : Register_Bank
port map (
    Reg_en => Reg_bus,
    Clk => slow_clock,
    Reset => Reset,
    Inp => Data_Mux_to_Reg_Bank,
    Q0 => Data_bus0,
    Q1 => Data_bus1,
    Q2 => Data_bus2,
    Q3 => Data_bus3,
    Q4 => Data_bus4,
    Q5 => Data_bus5,
    Q6 => Data_bus6,
    Q7 => Data_bus7
);

Add_Sub_Unit_0 : Add_Sub_Unit
port map (
    A => Mux_A_out,
    B => Mux_B_out,
    Sel => Add_Sub_Sel,
    En => '1',
    S => Add_Sub_Out,
    C_out => Carry_Flag,
    Z_out => Zero_Flag
);

MUX_2_way_4_Bit_0 : MUX_2_way_4_Bit
port map (
    Load => Load_Select,
    Adder_Sub_Out => Add_Sub_Out,
    Imd_Value => Imd_Val,
    Q => Data_Mux_to_Reg_Bank
);

MUX_8_way_4_Bit_A : MUX_8_way_4_Bit
port map (
    Reg_Sel => Reg_SelA,
    R0 => Data_bus0,

```

```

        R1 => Data_bus1,
        R2 => Data_bus2,
        R3 => Data_bus3,
        R4 => Data_bus4,
        R5 => Data_bus5,
        R6 => Data_bus6,
        R7 => Data_bus7,
        Q => Mux_A_out
    );

MUX_8_way_4_Bit_B : MUX_8_way_4_Bit
port map (
    Reg_Sel => Reg_SelB,
    R0 => Data_bus0,
    R1 => Data_bus1,
    R2 => Data_bus2,
    R3 => Data_bus3,
    R4 => Data_bus4,
    R5 => Data_bus5,
    R6 => Data_bus6,
    R7 => Data_bus7,
    Q => Mux_B_out
);

Adder_3_Bit_0 : Adder_3_Bit
port map (
    A => Mem_Bus,
    C_in => '0',
    S => Mem_RCA3_to_Mux,
    C_out => open
);

MUX_2_way_3_Bit_0 : MUX_2_way_3_Bit
port map (
    Jump => Jump_Flag,
    Adder_out => Mem_RCA3_to_Mux,
    Address => Jump_Add,
    Q => Count_Mux_to_PC
);

Overflow <= Carry_Flag;
Zero <= Zero_Flag;
R7 <= Data_bus7;

end Behavioral;

```

4.15.2 Simulation File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Nanoprocessor_Basic is
end TB_Nanoprocessor_Basic;

architecture Behavioral of TB_Nanoprocessor_Basic is

```

```

component Nanoprocessor_Basic
  Port (
    Clk      : in  STD_LOGIC;
    Reset    : in  STD_LOGIC;
    Overflow : out STD_LOGIC;
    Zero     : out STD_LOGIC;
    R7       : out STD_LOGIC_VECTOR(3 downto 0);
    Seg7_out : out STD_LOGIC_VECTOR(6 downto 0);
    Anode    : out STD_LOGIC_VECTOR(3 downto 0)
  );
end component;

signal Clk_TB      : STD_LOGIC := '0';
signal Reset_TB    : STD_LOGIC := '0';
signal Overflow_TB : STD_LOGIC;
signal Zero_TB     : STD_LOGIC;
signal R7_TB       : STD_LOGIC_VECTOR(3 downto 0);
signal Seg7_TB     : STD_LOGIC_VECTOR(6 downto 0);
signal Anode_TB    : STD_LOGIC_VECTOR(3 downto 0);

begin

  UUT : Nanoprocessor_Basic
    port map (
      Clk      => Clk_TB,
      Reset    => Reset_TB,
      Overflow => Overflow_TB,
      Zero     => Zero_TB,
      R7       => R7_TB,
      Seg7_out => Seg7_TB,
      Anode    => Anode_TB
    );

  Clk_TB <= not Clk_TB after 2 ns;

  process
  begin
    Reset_TB <= '1';
    wait for 30 ns;
    Reset_TB <= '0';

    wait for 500 ns;

    Reset_TB <= '1';
    wait for 30 ns;
    Reset_TB <= '0';

    wait for 500 ns;

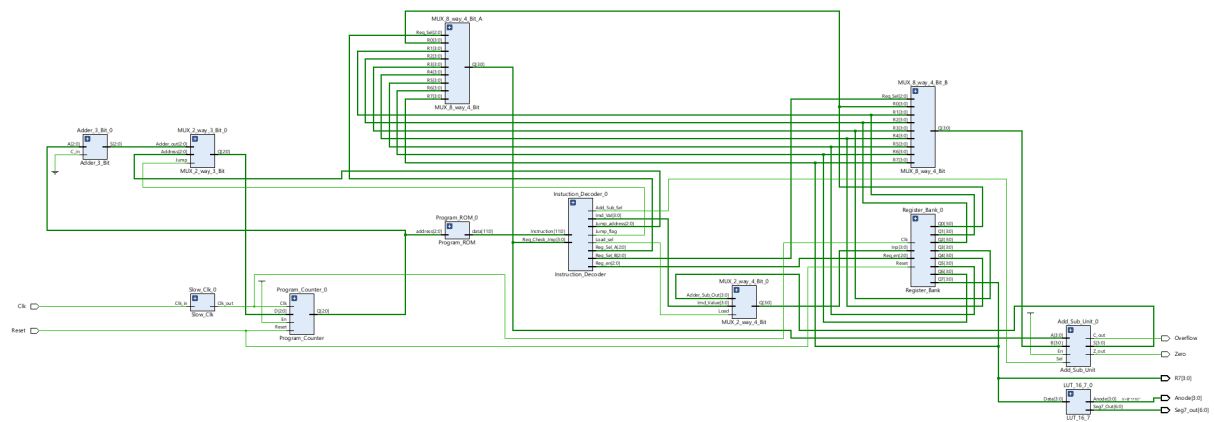
    Reset_TB <= '1';
    wait for 50 ns;

    wait;
  end process;

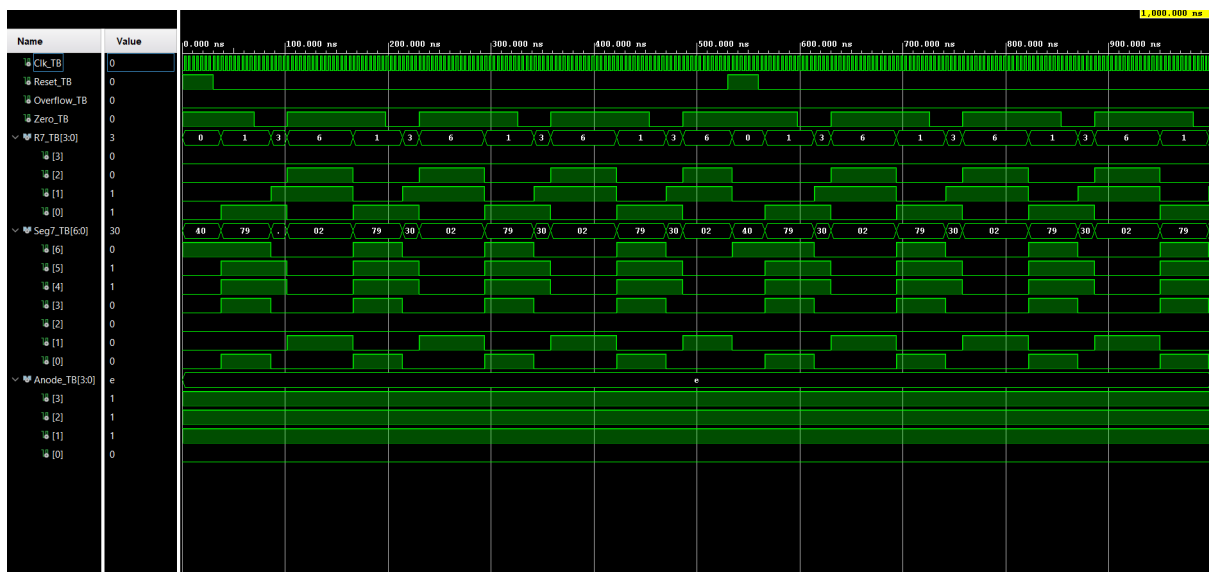
end Behavioral;

```

4.15.3 Elaborated design schematic



4.15.4 Timing Diagram



4.15.5 Constraint File

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports Clk]
set_property IOSTANDARD LVCMOS33 [get_ports Clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [
    get_ports Clk]

## Reset - Centre Button
set_property PACKAGE_PIN U18 [get_ports Reset]
set_property IOSTANDARD LVCMOS33 [get_ports Reset]

## R7 output - LED 0 to LED 3
set_property PACKAGE_PIN U16 [get_ports {R7[0]}]
```

```

set_property IOSTANDARD LVCMOS33 [get_ports {R7[0]}]
set_property PACKAGE_PIN E19 [get_ports {R7[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {R7[1]}]
set_property PACKAGE_PIN U19 [get_ports {R7[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {R7[2]}]
set_property PACKAGE_PIN V19 [get_ports {R7[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {R7[3]}]

## Zero flag - LED 14
set_property PACKAGE_PIN V14 [get_ports Zero]
set_property IOSTANDARD LVCMOS33 [get_ports Zero]

## Overflow flag - LED 15
set_property PACKAGE_PIN V3 [get_ports Overflow]
set_property IOSTANDARD LVCMOS33 [get_ports Overflow]

## 7 Segment Display - Segments
set_property PACKAGE_PIN W7 [get_ports {Seg7_out[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[0]}]
set_property PACKAGE_PIN W6 [get_ports {Seg7_out[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[1]}]
set_property PACKAGE_PIN U8 [get_ports {Seg7_out[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[2]}]
set_property PACKAGE_PIN V8 [get_ports {Seg7_out[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[3]}]
set_property PACKAGE_PIN U5 [get_ports {Seg7_out[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[4]}]
set_property PACKAGE_PIN V5 [get_ports {Seg7_out[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[5]}]
set_property PACKAGE_PIN U7 [get_ports {Seg7_out[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[6]}]

## 7 Segment Display - Anodes
set_property PACKAGE_PIN U2 [get_ports {Anode[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Anode[0]}]
set_property PACKAGE_PIN U4 [get_ports {Anode[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Anode[1]}]
set_property PACKAGE_PIN V4 [get_ports {Anode[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Anode[2]}]
set_property PACKAGE_PIN W4 [get_ports {Anode[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Anode[3]}]

```

5 Instructions to Operate Basic Nanoprocessor

5.1 Board Pin Mapping

5.1.1 Inputs

Component	Pin	Port	Description
Clock	W5	Clk	100 MHz onboard clock
Centre Button	U18	Reset	Reset & restart

5.1.2 Outputs – LEDs

LED	Pin	Signal	Description
LD0	U16	R7[0]	LSB
LD1	E19	R7[1]	
LD2	U19	R7[2]	
LD3	V19	R7[3]	
LD14	V14	Zero	ALU result = 0
LD15	V3	Overflow	Overflow flag

When R7 = 6 (0110)

LED	State
LD3	OFF
LD2	ON
LD1	ON
LD0	OFF

5.1.3 7-Segment Display

Segment	Pin	Signal
A	W7	Seg7_out[0]
B	W6	Seg7_out[1]
C	U8	Seg7_out[2]
D	V8	Seg7_out[3]
E	U5	Seg7_out[4]
F	V5	Seg7_out[5]
G	U7	Seg7_out[6]

5.1.4 Anodes

Anode	Pin	Description
an[0]	U2	Active digit (R7 output)
an[1]	U4	Unused
an[2]	V4	Unused
an[3]	W4	Unused

5.2 Program (ROM Instructions)

Addr	Instruction	Operation
0	MOVI R7, 1	R7 = 1
1	MOVI R5, 2	R5 = 2
2	MOVI R4, 3	R4 = 3
3	ADD R7, R5	R7 = 3
4	ADD R7, R4	R7 = 6
5–7	NOP	No operation

5.3 Execution Flow

Step	Output
After Instr 0	1
After Instr 3	3
After Instr 4	6

5.4 Final State (R7 = 6)

Component	State
7-Segment	6
LD3	OFF
LD2	ON
LD1	ON
LD0	OFF
Zero	OFF
Overflow	OFF

6 Additional Features

6.1 4-bit Comparator (Comparator_4_bit)

The 4-bit comparator compares two 4-bit values obtained from the outputs of MUX_A and MUX_B. It generates three output flags: `Equal_Flag`, `LessThan_Flag`, and `GreaterThan_Flag`. This component is useful for conditional branching operations, such as executing a jump instruction only if one register value is greater than another (e.g., jump if $R5 > R4$).

The testbench validates functionality by checking equal, less-than, and greater-than conditions, including boundary values such as 0 and 15.

6.2 4-bit Multiplier (Multiplier_4_bit)

The 4-bit multiplier multiplies two 4-bit operands using the VHDL `*` operator. The result is stored as a 4-bit output, and an additional `Overflow` flag is generated when the actual 8-bit product exceeds the 4-bit limit.

For example, $3 \times 2 = 6$ produces no overflow, whereas $4 \times 4 = 16$ exceeds 4-bit representation and sets the overflow flag. The testbench includes cases for zero multiplication, identity operations, normal multiplication, and overflow scenarios.

6.3 Parity Generator (Parity_Generator)

The parity generator computes both even and odd parity for the value stored in register R7 using a 4-input XOR chain. This is a purely combinational circuit and requires minimal hardware resources.

Even parity is set to '1' when the number of logic '1' bits is even (including zero), while odd parity is its complement. This feature is useful for demonstrating data integrity checking. For instance, when $R7 = 6$ (binary 0110), the number of '1's is two, resulting in `Even_Parity = '1'`.

6.4 Magnitude Detector (Magnitude_Detector)

The magnitude detector classifies the value in register R7 into three categories and activates the corresponding flag:

- `Is_Zero`: when the value is 0
- `Is_Small`: when the value is between 1 and 7
- `Is_Large`: when the value is between 8 and 15

This feature provides a quick visual indication of the value range using LEDs on the BASYS3 board. For example, when $R7 = 6$, the `Is_Small` flag is activated. The testbench verifies all categories, including boundary values such as 0, 7, and 8.

7 Advanced Nanoprocessor

7.1 4-bit Comparator

7.1.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Comparator_4_bit is
    Port (
        Input_A      : in  STD_LOGIC_VECTOR (3 downto 0);
        Input_B      : in  STD_LOGIC_VECTOR (3 downto 0);
        Equal_Flag    : out STD_LOGIC;
        LessThan_Flag : out STD_LOGIC;
        GreaterThan_Flag : out STD_LOGIC
    );
end Comparator_4_bit;

architecture Behavioral of Comparator_4_bit is
begin
    process(Input_A, Input_B)
    begin
        Equal_Flag      <= '0';
        LessThan_Flag   <= '0';
        GreaterThan_Flag <= '0';
        if (unsigned(Input_A) = unsigned(Input_B)) then
            Equal_Flag <= '1';
        elsif (unsigned(Input_A) < unsigned(Input_B)) then
            LessThan_Flag <= '1';
        else
            GreaterThan_Flag <= '1';
        end if;
    end process;
end Behavioral;
```

7.1.2 Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Comparator_4_bit is
end TB_Comparator_4_bit;

architecture Behavioral of TB_Comparator_4_bit is

    component Comparator_4_bit
        Port (
            Input_A      : in  STD_LOGIC_VECTOR (3 downto 0);
            Input_B      : in  STD_LOGIC_VECTOR (3 downto 0);
            Equal_Flag    : out STD_LOGIC;
            LessThan_Flag : out STD_LOGIC;
            GreaterThan_Flag : out STD_LOGIC
        );
    end component;
```

```

end component;

signal A_TB      : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
signal B_TB      : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
signal EQ_TB     : STD_LOGIC;
signal LT_TB     : STD_LOGIC;
signal GT_TB     : STD_LOGIC;

begin

    UUT : Comparator_4_bit
        port map (
            Input_A      => A_TB,
            Input_B      => B_TB,
            Equal_Flag    => EQ_TB,
            LessThan_Flag => LT_TB,
            GreaterThan_Flag => GT_TB
        );

    process
    begin
        A_TB <= "0000"; B_TB <= "0000"; wait for 100 ns;
        A_TB <= "0110"; B_TB <= "0110"; wait for 100 ns;
        A_TB <= "1111"; B_TB <= "1111"; wait for 100 ns;

        A_TB <= "0001"; B_TB <= "0010"; wait for 100 ns;
        A_TB <= "0011"; B_TB <= "0110"; wait for 100 ns;
        A_TB <= "0000"; B_TB <= "1111"; wait for 100 ns;

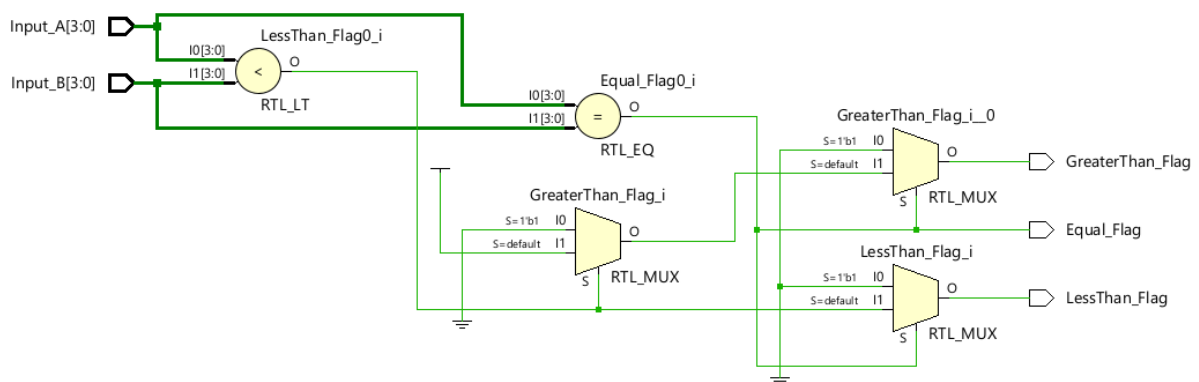
        A_TB <= "0110"; B_TB <= "0011"; wait for 100 ns;
        A_TB <= "1111"; B_TB <= "0001"; wait for 100 ns;
        A_TB <= "1000"; B_TB <= "0111"; wait for 100 ns;

        wait;
    end process;

end Behavioral;

```

7.1.3 Elaborated design schematic



7.1.4 Timing Diagram



7.2 4-bit Multiplier

7.2.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Multiplier_4_bit is
    Port (
        Input_A : in  STD_LOGIC_VECTOR (3 downto 0);
        Input_B : in  STD_LOGIC_VECTOR (3 downto 0);
        Result   : out STD_LOGIC_VECTOR (3 downto 0);
        Overflow : out STD_LOGIC
    );
end Multiplier_4_bit;

architecture Behavioral of Multiplier_4_bit is
    signal full_product : unsigned(7 downto 0);
begin
    full_product <= unsigned(Input_A) * unsigned(Input_B);
    Result <= std_logic_vector(full_product(3 downto 0));
    Overflow <= '1' when full_product(7 downto 4) /= "0000" else '0';
end Behavioral;
```

7.2.2 Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Multiplier_4_bit is
end TB_Multiplier_4_bit;

architecture Behavioral of TB_Multiplier_4_bit is

    component Multiplier_4_bit
        Port (
            Input_A : in  STD_LOGIC_VECTOR (3 downto 0);
            Input_B : in  STD_LOGIC_VECTOR (3 downto 0);
            Result   : out STD_LOGIC_VECTOR (3 downto 0);
            Overflow : out STD_LOGIC
        );
    end component;

    signal A_TB      : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal B_TB      : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal Result_TB  : STD_LOGIC_VECTOR(3 downto 0);
    signal OV_TB      : STD_LOGIC;

begin

    UUT : Multiplier_4_bit
        port map (
            Input_A => A_TB,
            Input_B => B_TB,
            Result  => Result_TB,
```

```

        Overflow => OV_TB
    );

process
begin
    A_TB <= "0000"; B_TB <= "0000"; wait for 100 ns;
    A_TB <= "0001"; B_TB <= "0001"; wait for 100 ns;

    A_TB <= "0010"; B_TB <= "0011"; wait for 100 ns;
    A_TB <= "0011"; B_TB <= "0010"; wait for 100 ns;
    A_TB <= "0110"; B_TB <= "0001"; wait for 100 ns;

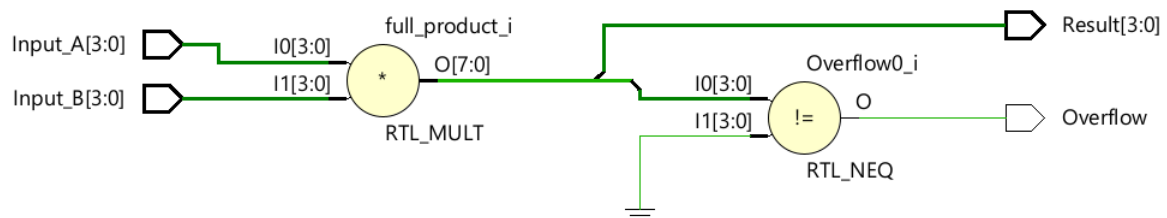
    A_TB <= "0011"; B_TB <= "0011"; wait for 100 ns;
    A_TB <= "0100"; B_TB <= "0100"; wait for 100 ns;
    A_TB <= "1111"; B_TB <= "1111"; wait for 100 ns;
    A_TB <= "1000"; B_TB <= "0010"; wait for 100 ns;

    wait;
end process;

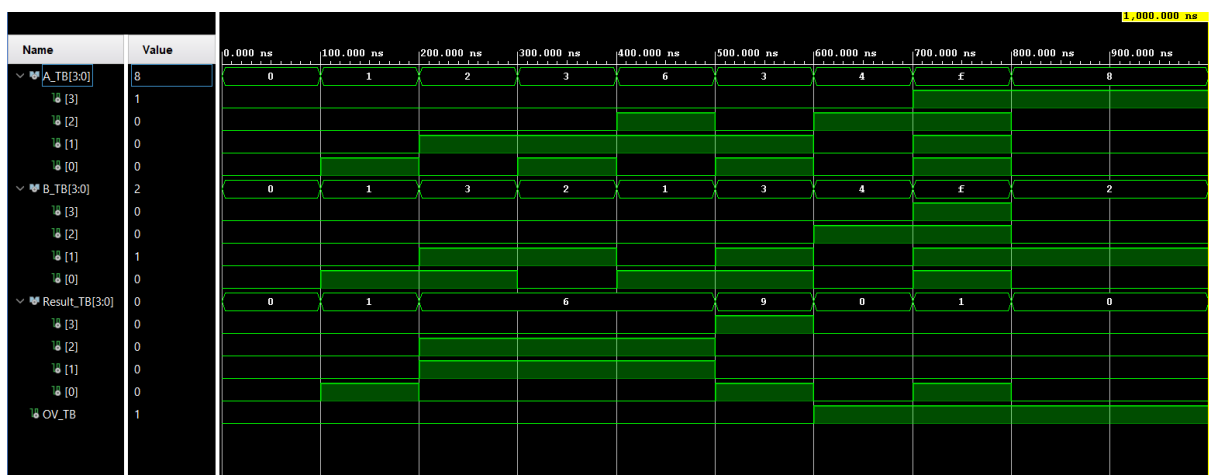
end Behavioral;

```

7.2.3 Elaborated design schematic



7.2.4 Timing Diagram



7.3 Parity Generator

7.3.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Parity_Generator is
    Port (
        Data_In      : in  STD_LOGIC_VECTOR (3 downto 0);
        Even_Parity   : out STD_LOGIC;
        Odd_Parity    : out STD_LOGIC
    );
end Parity_Generator;

architecture Behavioral of Parity_Generator is
    signal xor_chain : STD_LOGIC;
begin
    xor_chain <= Data_In(3) XOR Data_In(2) XOR Data_In(1) XOR Data_In(0);
    Even_Parity <= NOT xor_chain;
    Odd_Parity  <= xor_chain;
end Behavioral;
```

7.3.2 Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Parity_Generator is
end TB_Parity_Generator;

architecture Behavioral of TB_Parity_Generator is

    component Parity_Generator
        Port (
            Data_In      : in  STD_LOGIC_VECTOR (3 downto 0);
            Even_Parity   : out STD_LOGIC;
            Odd_Parity    : out STD_LOGIC
        );
    end component;

    signal Data_TB  : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal Even_TB   : STD_LOGIC;
    signal Odd_TB    : STD_LOGIC;

begin

    UUT : Parity_Generator
        port map (
            Data_In      => Data_TB,
            Even_Parity  => Even_TB,
            Odd_Parity   => Odd_TB
        );

    process
    begin
```

```

Data_TB <= "0000"; wait for 100 ns;
Data_TB <= "1111"; wait for 100 ns;

Data_TB <= "0001"; wait for 100 ns;
Data_TB <= "0011"; wait for 100 ns;
Data_TB <= "0111"; wait for 100 ns;
Data_TB <= "1110"; wait for 100 ns;

Data_TB <= "0110"; wait for 100 ns;
Data_TB <= "1001"; wait for 100 ns;
Data_TB <= "1010"; wait for 100 ns;
Data_TB <= "0101"; wait for 100 ns;

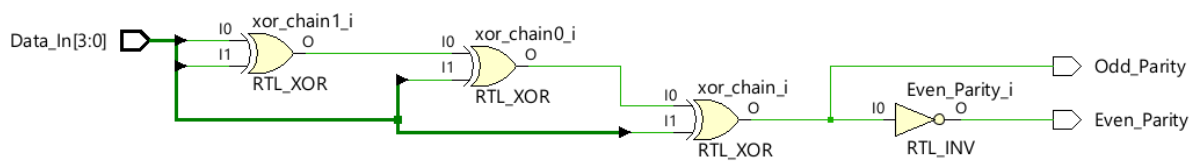
Data_TB <= "1000"; wait for 100 ns;
Data_TB <= "0100"; wait for 100 ns;

wait;
end process;

end Behavioral;

```

7.3.3 Elaborated design schematic



7.3.4 Timing Diagram



7.4 Magnitude Detector

7.4.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Magnitude_Detector is
    Port (
        Data_In      : in  STD_LOGIC_VECTOR (3 downto 0);
        Is_Zero       : out STD_LOGIC;
        Is_Small      : out STD_LOGIC;
        Is_Large      : out STD_LOGIC
    );
end Magnitude_Detector;

architecture Behavioral of Magnitude_Detector is
begin
    process (Data_In)
    begin
        Is_Zero  <= '0';
        Is_Small <= '0';
        Is_Large <= '0';
        if unsigned(Data_In) = 0 then
            Is_Zero <= '1';
        elsif unsigned(Data_In) <= 7 then
            Is_Small <= '1';
        else
            Is_Large <= '1';
        end if;
    end process;
end Behavioral;
```

7.4.2 Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Magnitude_Detector is
end TB_Magnitude_Detector;

architecture Behavioral of TB_Magnitude_Detector is

    component Magnitude_Detector
        Port (
            Data_In      : in  STD_LOGIC_VECTOR (3 downto 0);
            Is_Zero       : out STD_LOGIC;
            Is_Small      : out STD_LOGIC;
            Is_Large      : out STD_LOGIC
        );
    end component;

    signal Data_TB      : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal Zero_TB       : STD_LOGIC;
    signal Small_TB      : STD_LOGIC;
```

```

signal Large_TB    : STD_LOGIC;

begin

    UUT : Magnitude_Detector
        port map (
            Data_In  => Data_TB,
            Is_Zero  => Zero_TB,
            Is_Small => Small_TB,
            Is_Large => Large_TB
        );

    process
    begin
        Data_TB <= "0000"; wait for 100 ns;

        Data_TB <= "0001"; wait for 100 ns;
        Data_TB <= "0011"; wait for 100 ns;
        Data_TB <= "0110"; wait for 100 ns;
        Data_TB <= "0111"; wait for 100 ns;

        Data_TB <= "1000"; wait for 100 ns;
        Data_TB <= "1010"; wait for 100 ns;
        Data_TB <= "1110"; wait for 100 ns;
        Data_TB <= "1111"; wait for 100 ns;

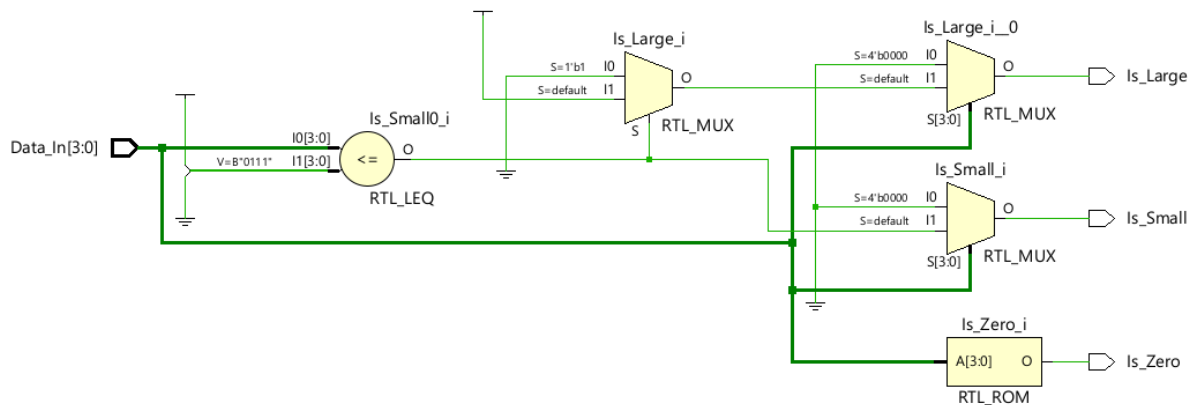
        Data_TB <= "0000"; wait for 100 ns;
        Data_TB <= "0101"; wait for 100 ns;
        Data_TB <= "1100"; wait for 100 ns;

        wait;
    end process;

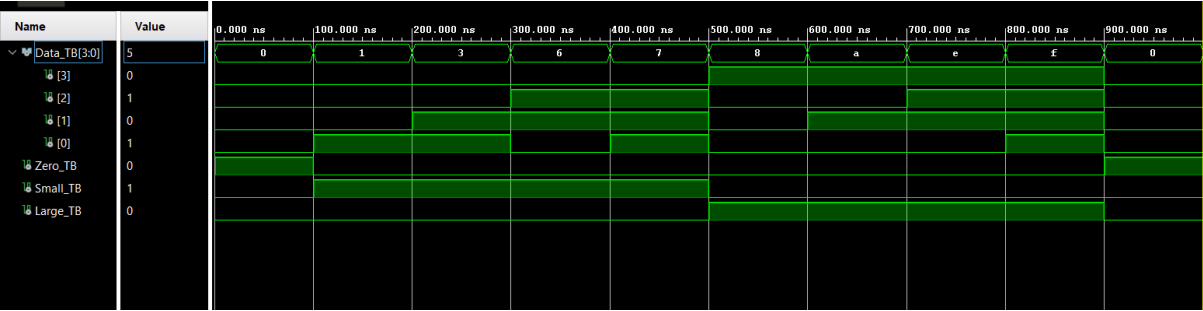
end Behavioral;

```

7.4.3 Elaborated design schematic



7.4.4 Timing Diagram



7.5 Instruction Decoder

7.5.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Instuction_Decoder is
    Port (
        Instruction      : in  STD_LOGIC_VECTOR(11 downto 0);
        Reg_Check_Jmp    : in  STD_LOGIC_VECTOR(3  downto 0);
        Reg_en           : out STD_LOGIC_VECTOR(2  downto 0);
        Load_sel         : out STD_LOGIC;
        Imd_Val          : out STD_LOGIC_VECTOR(3  downto 0);
        Reg_Sel_A         : out STD_LOGIC_VECTOR(2  downto 0);
        Reg_Sel_B         : out STD_LOGIC_VECTOR(2  downto 0);
        Operation_Select  : out STD_LOGIC_VECTOR(2  downto 0);
        Jump_address     : out STD_LOGIC_VECTOR(2  downto 0);
        Jump_flag        : out STD_LOGIC
    );
end Instuction_Decoder;

architecture Behavioral of Instuction_Decoder is
begin

    process(Instruction, Reg_Check_Jmp)
        variable Opcode : STD_LOGIC_VECTOR(1  downto 0);
        variable SubOp  : STD_LOGIC_VECTOR(3  downto 0);
    begin
        Opcode := Instruction(11 downto 10);
        SubOp  := Instruction(3  downto 0);

        Jump_flag      <= '0';
        Reg_en         <= "000";
        Load_sel       <= '0';
        Operation_Select <= "000";
        Reg_Sel_A      <= Instruction(9  downto 7);
        Reg_Sel_B      <= Instruction(6  downto 4);
        Imd_Val        <= Instruction(3  downto 0);
        Jump_address   <= Instruction(2  downto 0);

        case Opcode is

            when "10" =>
                Reg_en  <= Instruction(9  downto 7);
                Load_sel <= '1';

            when "00" =>
                Reg_en  <= Instruction(9  downto 7);
                Load_sel <= '0';
                case SubOp is
                    when "0000" => Operation_Select <= "000";
                    when "0001" => Operation_Select <= "001";
                    when "0101" => Operation_Select <= "101";
                    when "0110" => Operation_Select <= "110";
                    when others => Operation_Select <= "000";
                end case;

        end case;

    end process;

end Behavioral;
```

```

when "01" =>
    Reg_en          <= Instruction(9 downto 7);
    Load_sel       <= '0';
    Operation_Select <= "111";

when "11" =>
    if Reg_Check_Jmp = "0000" then
        Jump_flag <= '1';
    end if;

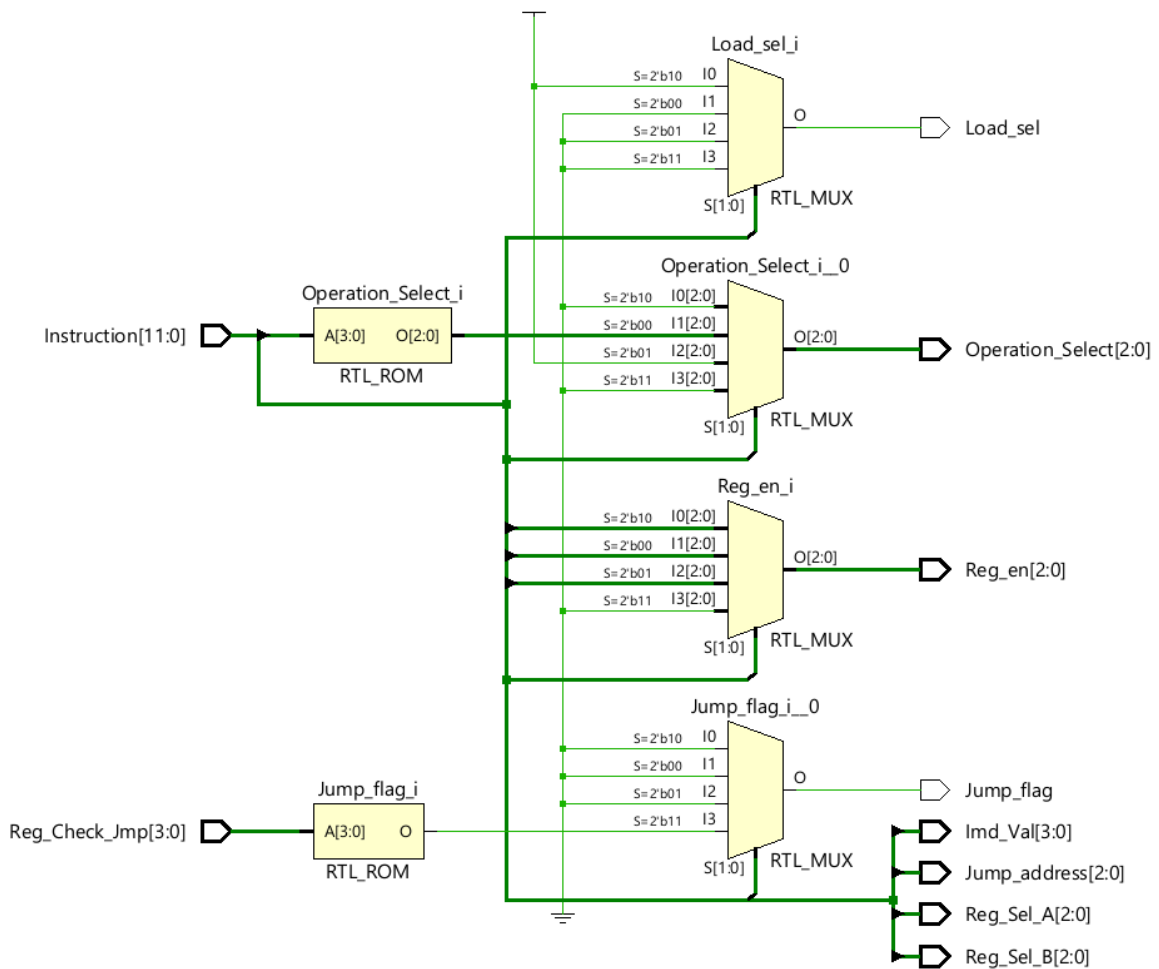
when others => null;

end case;
end process;

end Behavioral;

```

7.5.2 Elaborated design schematic



7.6 ALU

7.6.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ALU is
    Port (
        A, B          : in  STD_LOGIC_VECTOR(3 downto 0);
        Operation      : in  STD_LOGIC_VECTOR(2 downto 0);
        Result         : out STD_LOGIC_VECTOR(3 downto 0);
        Zero           : out STD_LOGIC;
        Overflow       : out STD_LOGIC;
        Equal          : out STD_LOGIC;
        LessThan       : out STD_LOGIC;
        GreaterThan    : out STD_LOGIC
    );
end ALU;

architecture Behavioral of ALU is

    component Add_Sub_4_bit
        Port (
            Input_A, Input_B : in  STD_LOGIC_VECTOR(3 downto 0);
            Mode_Sel         : in  STD_LOGIC;
            Result           : out STD_LOGIC_VECTOR(3 downto 0);
            Overflow_Flag    : out STD_LOGIC
        );
    end component;

    component Comparator_4_bit
        Port (
            Input_A, Input_B : in  STD_LOGIC_VECTOR(3 downto 0);
            Equal_Flag       : out STD_LOGIC;
            LessThan_Flag    : out STD_LOGIC;
            GreaterThan_Flag : out STD_LOGIC
        );
    end component;

    component Multiplier_4_bit
        Port (
            Input_A, Input_B : in  STD_LOGIC_VECTOR(3 downto 0);
            Result           : out STD_LOGIC_VECTOR(3 downto 0);
            Overflow         : out STD_LOGIC
        );
    end component;

    signal AS_Res, Mul_Res : STD_LOGIC_VECTOR(3 downto 0);
    signal Sel_A, Sel_B    : STD_LOGIC_VECTOR(3 downto 0);
    signal AS_Mode         : STD_LOGIC;
    signal Mul_OV_internal : STD_LOGIC;

begin

    Sel_A <= "0000" when Operation = "111" else A;
    Sel_B <= A      when Operation = "111" else B;
```

```

AS_Mode <= '1'      when (Operation = "001" or Operation = "111") else
'0';

AS_Unit : Add_Sub_4_bit
  port map (
    Input_A      => Sel_A,
    Input_B      => Sel_B,
    Mode_Sel     => AS_Mode,
    Result       => AS_Res,
    Overflow_Flag => Overflow
  );

Comp_Unit : Comparator_4_bit
  port map (
    Input_A      => A,
    Input_B      => B,
    Equal_Flag   => Equal,
    LessThan_Flag => LessThan,
    GreaterThan_Flag => GreaterThan
  );

Mul_Unit : Multiplier_4_bit
  port map (
    Input_A => A,
    Input_B => B,
    Result  => Mul_Res,
    Overflow => Mul_OV_internal
  );

process(Operation, AS_Res, Mul_Res, A, B)
begin
  case Operation is
    when "000" | "001" | "111" => Result <= AS_Res;
    when "101"                  => Result <= Mul_Res;
    when "010"                  => Result <= A and B;
    when "011"                  => Result <= A or B;
    when "100"                  => Result <= A xor B;
    when others                  => Result <= "0000";
  end case;
end process;

Zero <= '1' when AS_Res = "0000" else '0';

end Behavioral;

```

7.7 Advanced Nanoprocessor

7.7.1 Design File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Nanoprocessor_Advance is
    Port (
        Clk          : in  STD_LOGIC;
        Reset         : in  STD_LOGIC;
        Overflow      : out STD_LOGIC;
        Zero          : out STD_LOGIC;
        Equal         : out STD_LOGIC;
        Less          : out STD_LOGIC;
        Greater       : out STD_LOGIC;
        Even_Parity   : out STD_LOGIC;
        Odd_Parity    : out STD_LOGIC;
        Is_Zero       : out STD_LOGIC;
        Is_Small      : out STD_LOGIC;
        Is_Large      : out STD_LOGIC;
        Mult_OV       : out STD_LOGIC;
        Mult_Result    : out STD_LOGIC;
        R7            : out STD_LOGIC_VECTOR(3 downto 0);
        Seg7_out      : out STD_LOGIC_VECTOR(6 downto 0);
        Anode         : out STD_LOGIC_VECTOR(3 downto 0)
    );
end Nanoprocessor_Advance;

architecture Behavioral of Nanoprocessor_Advance is

    component LUT_16_7
        Port (
            Data      : in  STD_LOGIC_VECTOR(3 downto 0);
            Seg7_out   : out STD_LOGIC_VECTOR(6 downto 0);
            Anode      : out STD_LOGIC_VECTOR(3 downto 0)
        );
    end component;

    component Slow_Clk
        Port (
            Clk_in    : in  STD_LOGIC;
            Clk_out    : out STD_LOGIC
        );
    end component;

    component Register_Bank
        Port (
            Reg_en          : in  STD_LOGIC_VECTOR(2 downto 0);
            Clk              : in  STD_LOGIC;
            Reset            : in  STD_LOGIC;
            Inp              : in  STD_LOGIC_VECTOR(3 downto 0);
            Q0,Q1,Q2,Q3,Q4,Q5,Q6,Q7 : out STD_LOGIC_VECTOR(3 downto 0)
        );
    end component;

    component MUX_8_way_4_Bit
        Port (

```

```

        Reg_Sel          : in  STD_LOGIC_VECTOR(2 downto 0);
        R0,R1,R2,R3,R4,R5,R6,R7 : in  STD_LOGIC_VECTOR(3 downto 0);
        Q                 : out STD_LOGIC_VECTOR(3 downto 0)
    );
end component;

component ALU
    Port (
        A, B              : in  STD_LOGIC_VECTOR(3 downto 0);
        Operation          : in  STD_LOGIC_VECTOR(2 downto 0);
        Result              : out STD_LOGIC_VECTOR(3 downto 0);
        Zero                : out STD_LOGIC;
        Overflow            : out STD_LOGIC;
        Equal               : out STD_LOGIC;
        LessThan            : out STD_LOGIC;
        GreaterThan         : out STD_LOGIC
    );
end component;

component MUX_2_way_4_Bit
    Port (
        Adder_Sub_Out      : in  STD_LOGIC_VECTOR(3 downto 0);
        Imd_Value           : in  STD_LOGIC_VECTOR(3 downto 0);
        Load                : in  STD_LOGIC;
        Q                   : out STD_LOGIC_VECTOR(3 downto 0)
    );
end component;

component MUX_2_way_3_Bit
    Port (
        Adder_out           : in  STD_LOGIC_VECTOR(2 downto 0);
        Address              : in  STD_LOGIC_VECTOR(2 downto 0);
        Jump                 : in  STD_LOGIC;
        Q                    : out STD_LOGIC_VECTOR(2 downto 0)
    );
end component;

component Instuction_Decoder
    Port (
        Instruction          : in  STD_LOGIC_VECTOR(11 downto 0);
        Reg_Check_Jmp        : in  STD_LOGIC_VECTOR(3 downto 0);
        Reg_en               : out STD_LOGIC_VECTOR(2 downto 0);
        Load_sel             : out STD_LOGIC;
        Imd_Val              : out STD_LOGIC_VECTOR(3 downto 0);
        Reg_Sel_A             : out STD_LOGIC_VECTOR(2 downto 0);
        Reg_Sel_B             : out STD_LOGIC_VECTOR(2 downto 0);
        Operation_Select     : out STD_LOGIC_VECTOR(2 downto 0);
        Jump_address         : out STD_LOGIC_VECTOR(2 downto 0);
        Jump_flag            : out STD_LOGIC
    );
end component;

component Program_ROM
    Port (
        address : in  STD_LOGIC_VECTOR(2 downto 0);
        data     : out STD_LOGIC_VECTOR(11 downto 0)
    );
end component;

```

```

component Program_Counter
  Port (
    D      : in  STD_LOGIC_VECTOR(2 downto 0);
    En     : in  STD_LOGIC;
    Reset  : in  STD_LOGIC;
    Clk    : in  STD_LOGIC;
    Q      : out STD_LOGIC_VECTOR(2 downto 0)
  );
end component;

component Adder_3_Bit
  Port (
    A      : in  STD_LOGIC_VECTOR(2 downto 0);
    C_in   : in  STD_LOGIC;
    S      : out STD_LOGIC_VECTOR(2 downto 0);
    C_out  : out STD_LOGIC
  );
end component;

component Multiplier_4_bit
  Port (
    Input_A : in  STD_LOGIC_VECTOR(3 downto 0);
    Input_B : in  STD_LOGIC_VECTOR(3 downto 0);
    Result  : out STD_LOGIC_VECTOR(3 downto 0);
    Overflow : out STD_LOGIC
  );
end component;

component Parity_Generator
  Port (
    Data_In      : in  STD_LOGIC_VECTOR(3 downto 0);
    Even_Parity  : out STD_LOGIC;
    Odd_Parity   : out STD_LOGIC
  );
end component;

component Magnitude_Detector
  Port (
    Data_In : in  STD_LOGIC_VECTOR(3 downto 0);
    Is_Zero : out STD_LOGIC;
    Is_Small : out STD_LOGIC;
    Is_Large : out STD_LOGIC
  );
end component;

signal Imd_Val_sig, ALU_Out_Bus, Data_Mux_to_Reg_Bank :
STD_LOGIC_VECTOR(3 downto 0);
signal Data_bus0, Data_bus1, Data_bus2, Data_bus3,
      Data_bus4, Data_bus5, Data_bus6, Data_bus7 :
STD_LOGIC_VECTOR(3 downto 0);
signal Ins_Bus :
STD_LOGIC_VECTOR(11 downto 0);
signal Mem_Bus, Count_Mux_to_PC,
      Mem_RCA3_to_Mux, Jump_Add :
STD_LOGIC_VECTOR(2 downto 0);
signal Reg_bus, Reg_SelA, Reg_SelB, Op_Sel :
STD_LOGIC_VECTOR(2 downto 0);

```

```

    signal Load_Select, Jump_Flag, slow_clock           : STD_LOGIC;
    signal Mux_A_out, Mux_B_out                         :
STD_LOGIC_VECTOR(3 downto 0);
    signal Carry_Flag, Zero_Flag,
           Eq_Flag, Lt_Flag, Gt_Flag                   : STD_LOGIC;
    signal Mult_Result_internal                         :
STD_LOGIC_VECTOR(3 downto 0);
    signal Mult_OV_internal                            : STD_LOGIC;
    signal Odd_Parity_internal                         : STD_LOGIC;
    signal Is_Large_internal                           : STD_LOGIC;

begin

    LUT_16_7_0 : LUT_16_7
        port map (
            Data      => Data_bus7,
            Seg7_out  => Seg7_out,
            Anode     => Anode
        );

    Slow_Clk_0 : Slow_Clk
        port map (
            Clk_in   => Clk,
            Clk_out  => slow_clock
        );

    ROM_0 : Program_ROM
        port map (
            address => Mem_Bus,
            data    => Ins_Bus
        );

    Program_Counter_0 : Program_Counter
        port map (
            D      => Count_Mux_to_PC,
            En     => '1',
            Reset  => Reset,
            Clk    => slow_clock,
            Q      => Mem_Bus
        );

    Instuction_Decoder_0 : Instuction_Decoder
        port map (
            Instruction      => Ins_Bus,
            Reg_en           => Reg_bus,
            Load_sel        => Load_Select,
            Imd_Val          => Imd_Val_sig,
            Reg_Sel_A        => Reg_SelA,
            Reg_Sel_B        => Reg_SelB,
            Operation_Select => Op_Sel,
            Jump_address     => Jump_Add,
            Jump_flag        => Jump_Flag,
            Reg_Check_Jmp    => Mux_A_out
        );

    Register_Bank_0 : Register_Bank
        port map (
            Reg_en => Reg_bus,

```

```

        Clk      => slow_clock,
        Reset    => Reset,
        Inp      => Data_Mux_to_Reg_Bank,
        Q0       => Data_bus0, Q1 => Data_bus1,
        Q2       => Data_bus2, Q3 => Data_bus3,
        Q4       => Data_bus4, Q5 => Data_bus5,
        Q6       => Data_bus6, Q7 => Data_bus7
    );

ALU_0 : ALU
    port map (
        A          => Mux_A_out,
        B          => Mux_B_out,
        Operation  => Op_Sel,
        Result     => ALU_Out_Bus,
        Zero       => Zero_Flag,
        Overflow   => Carry_Flag,
        Equal      => Eq_Flag,
        LessThan   => Lt_Flag,
        GreaterThan => Gt_Flag
    );

MUX_2_way_4_Bit_0 : MUX_2_way_4_Bit
    port map (
        Load       => Load_Select,
        Adder_Sub_Out => ALU_Out_Bus,
        Imd_Value   => Imd_Val_sig,
        Q           => Data_Mux_to_Reg_Bank
    );

MUX_8_way_4_Bit_A : MUX_8_way_4_Bit
    port map (
        Reg_Sel => Reg_SelA,
        R0 => Data_bus0, R1 => Data_bus1,
        R2 => Data_bus2, R3 => Data_bus3,
        R4 => Data_bus4, R5 => Data_bus5,
        R6 => Data_bus6, R7 => Data_bus7,
        Q  => Mux_A_out
    );

MUX_8_way_4_Bit_B : MUX_8_way_4_Bit
    port map (
        Reg_Sel => Reg_SelB,
        R0 => Data_bus0, R1 => Data_bus1,
        R2 => Data_bus2, R3 => Data_bus3,
        R4 => Data_bus4, R5 => Data_bus5,
        R6 => Data_bus6, R7 => Data_bus7,
        Q  => Mux_B_out
    );

Adder_3_Bit_0 : Adder_3_Bit
    port map (
        A      => Mem_Bus,
        C_in   => '0',
        S      => Mem_RCA3_to_Mux,
        C_out  => open
    );

```

```

MUX_2_way_3_Bit_0 : MUX_2_way_3_Bit
  port map (
    Jump      => Jump_Flag,
    Adder_out => Mem_RCA3_to_Mux,
    Address   => Jump_Add,
    Q         => Count_Mux_to_PC
  );

Multiplier_top : Multiplier_4_bit
  port map (
    Input_A  => Mux_A_out,
    Input_B  => Mux_B_out,
    Result   => Mult_Result_internal,
    Overflow => Mult_OV_internal
  );

Parity_Generator_0 : Parity_Generator
  port map (
    Data_In      => Data_bus7,
    Even_Parity  => Even_Parity,
    Odd_Parity   => Odd_Parity_internal
  );

Magnitude_Detector_0 : Magnitude_Detector
  port map (
    Data_In  => Data_bus7,
    Is_Zero  => Is_Zero,
    Is_Small => Is_Small,
    Is_Large => Is_Large_internal
  );

Overflow    <= Carry_Flag;
Zero        <= Zero_Flag;
Equal       <= Eq_Flag;
Less        <= Lt_Flag;
Greater     <= Gt_Flag;
R7          <= Data_bus7;

Mult_Result <= Mult_Result_internal(0);
Mult_OV     <= Mult_OV_internal;

Odd_Parity  <= Odd_Parity_internal;

Is_Large    <= Is_Large_internal;

end Behavioral;

```

7.7.2 Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Nanoprocessor_Advance is
end TB_Nanoprocessor_Advance;

architecture Behavioral of TB_Nanoprocessor_Advance is

```

```

component Nanoprocessor_Advance
  Port (
    Clk          : in  STD_LOGIC;
    Reset        : in  STD_LOGIC;
    Overflow      : out STD_LOGIC;
    Zero          : out STD_LOGIC;
    Equal         : out STD_LOGIC;
    Less         : out STD_LOGIC;
    Greater       : out STD_LOGIC;
    Mult_Result   : out STD_LOGIC_VECTOR (3 downto 0);
    Mult_OV       : out STD_LOGIC;
    Even_Parity   : out STD_LOGIC;
    Odd_Parity    : out STD_LOGIC;
    Is_Zero       : out STD_LOGIC;
    Is_Small      : out STD_LOGIC;
    Is_Large      : out STD_LOGIC;
    R7            : out STD_LOGIC_VECTOR (3 downto 0);
    Seg7_out      : out STD_LOGIC_VECTOR (6 downto 0);
    Anode         : out STD_LOGIC_VECTOR (3 downto 0)
  );
end component;

signal Clk_TB          : STD_LOGIC := '0';
signal Reset_TB        : STD_LOGIC := '1';
signal Overflow_TB     : STD_LOGIC;
signal Zero_TB         : STD_LOGIC;
signal Equal_TB        : STD_LOGIC;
signal Less_TB         : STD_LOGIC;
signal Greater_TB      : STD_LOGIC;
signal Mult_Result_TB  : STD_LOGIC_VECTOR(3 downto 0);
signal Mult_OV_TB      : STD_LOGIC;
signal Even_TB         : STD_LOGIC;
signal Odd_TB          : STD_LOGIC;
signal IsZero_TB       : STD_LOGIC;
signal IsSmall_TB      : STD_LOGIC;
signal IsLarge_TB      : STD_LOGIC;
signal R7_TB           : STD_LOGIC_VECTOR(3 downto 0);
signal Seg7_TB         : STD_LOGIC_VECTOR(6 downto 0);
signal Anode_TB        : STD_LOGIC_VECTOR(3 downto 0);

constant CLK_PERIOD : time := 10 ns;
constant SLOW_TICK  : time := 40 ns;

begin

  UUT : Nanoprocessor_Advance
    port map (
      Clk          => Clk_TB,
      Reset        => Reset_TB,
      Overflow      => Overflow_TB,
      Zero          => Zero_TB,
      Equal         => Equal_TB,
      Less         => Less_TB,
      Greater       => Greater_TB,
      Mult_Result   => Mult_Result_TB,
      Mult_OV       => Mult_OV_TB,
      Even_Parity   => Even_TB,

```

```

        Odd_Parity  => Odd_TB,
        Is_Zero     => IsZero_TB,
        Is_Small    => IsSmall_TB,
        Is_Large    => IsLarge_TB,
        R7          => R7_TB,
        Seg7_out    => Seg7_TB,
        Anode       => Anode_TB
    );

    Clk_TB <= not Clk_TB after CLK_PERIOD / 2;

process
begin

    Reset_TB <= '1';
    wait for 100 ns;
    Reset_TB <= '0';
    wait for SLOW_TICK * 2;

    wait for SLOW_TICK * 8;

    Reset_TB <= '1';
    wait for 50 ns;
    Reset_TB <= '0';
    wait for SLOW_TICK * 8;

    Reset_TB <= '1';
    wait for 50 ns;
    Reset_TB <= '0';
    wait for SLOW_TICK * 8;

    wait for SLOW_TICK * 24;

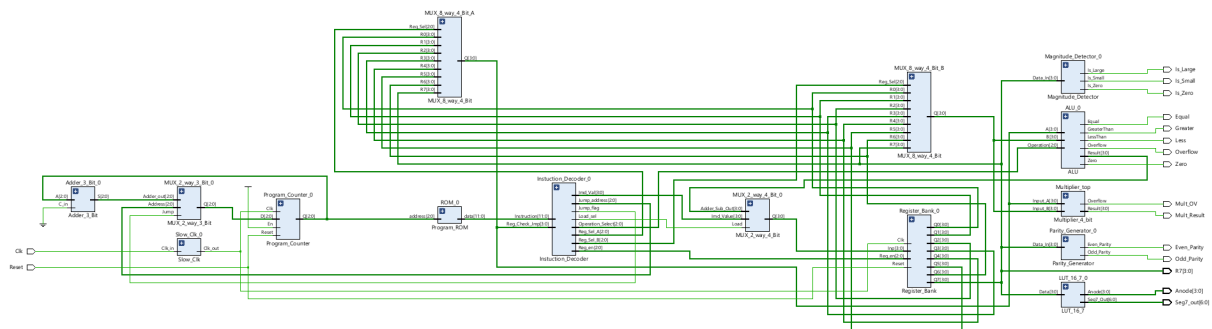
    Reset_TB <= '1';
    wait for 50 ns;
    Reset_TB <= '0';
    wait for SLOW_TICK * 10;

    wait;
end process;

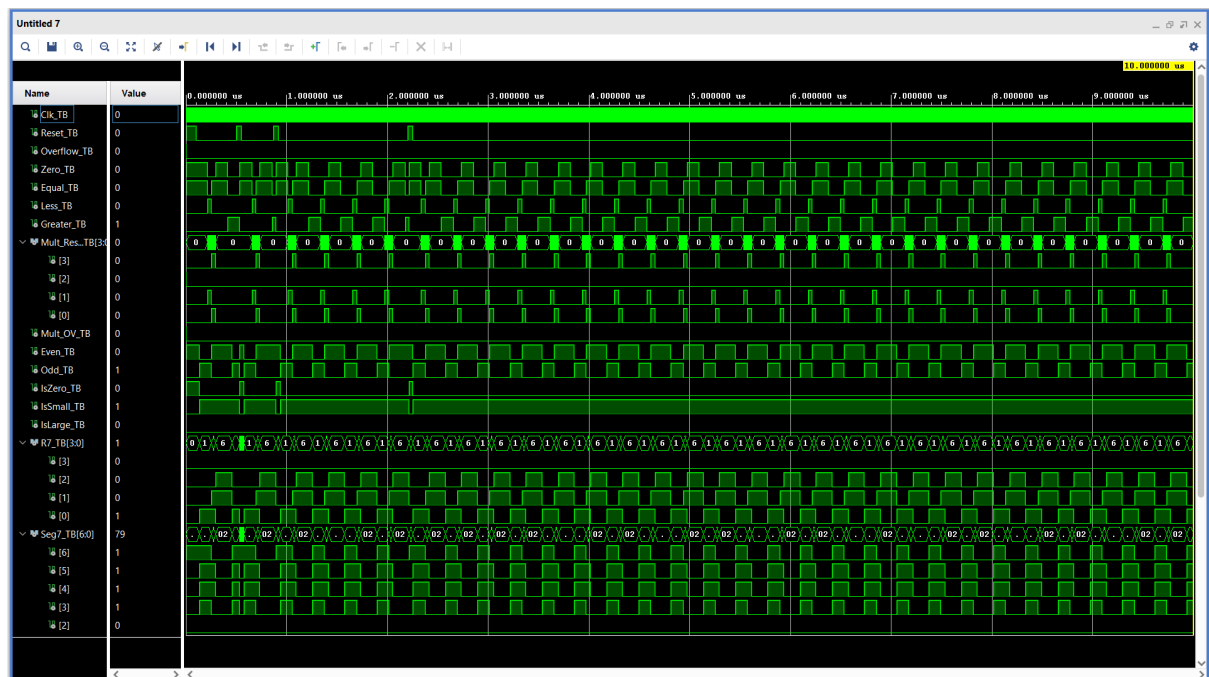
end Behavioral;

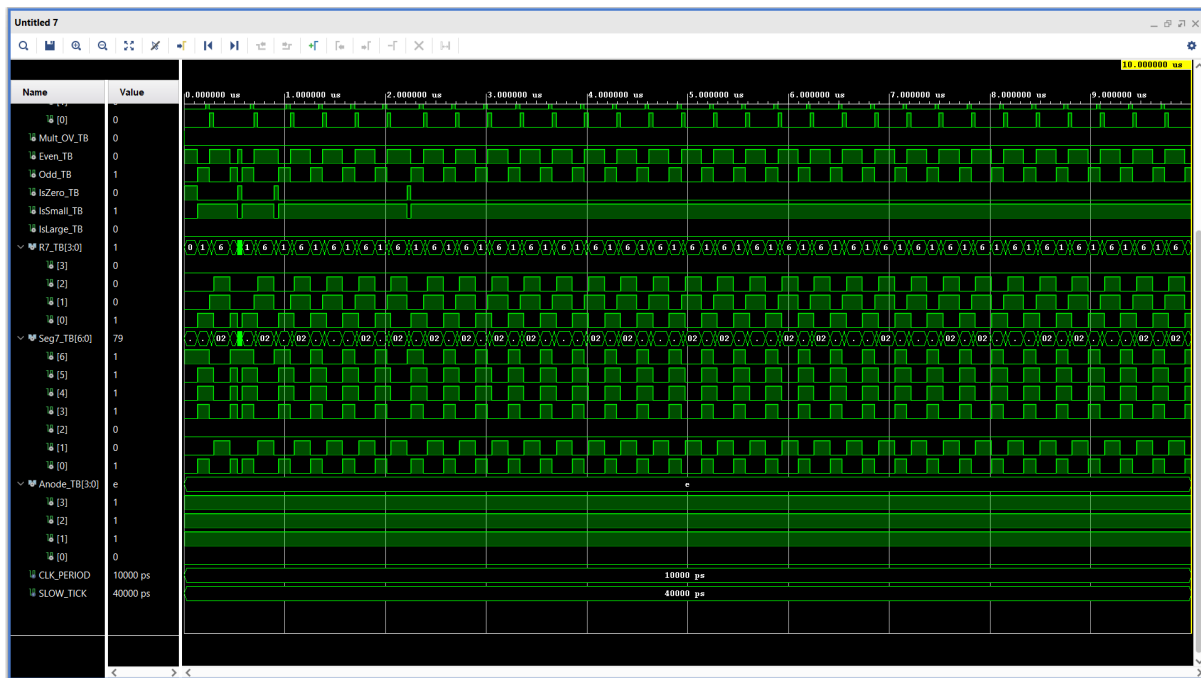
```

7.7.3 Elaborated design schematic



7.7.4 Timing Diagram





7.7.5 Constraint File

```
## -----
## Clock
## -----
set_property PACKAGE_PIN W5 [get_ports Clk]
    set_property IOSTANDARD LVCMOS33 [get_ports Clk]
    create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [
get_ports Clk]

## -----
## Reset
## -----
set_property PACKAGE_PIN U18 [get_ports Reset]
    set_property IOSTANDARD LVCMOS33 [get_ports Reset]

## -----
## 7-Segment Display - segments (active low)
## -----
set_property PACKAGE_PIN W7 [get_ports {Seg7_out[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[0]}]
set_property PACKAGE_PIN W6 [get_ports {Seg7_out[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[1]}]
set_property PACKAGE_PIN U8 [get_ports {Seg7_out[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[2]}]
set_property PACKAGE_PIN V8 [get_ports {Seg7_out[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[3]}]
set_property PACKAGE_PIN U5 [get_ports {Seg7_out[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[4]}]
set_property PACKAGE_PIN V5 [get_ports {Seg7_out[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[5]}]
set_property PACKAGE_PIN U7 [get_ports {Seg7_out[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Seg7_out[6]}]

## -----
```

```

## 7-Segment Display - anodes (active low)
## -----
set_property PACKAGE_PIN U2 [get_ports {Anode[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Anode[0]}]
set_property PACKAGE_PIN U4 [get_ports {Anode[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Anode[1]}]
set_property PACKAGE_PIN V4 [get_ports {Anode[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Anode[2]}]
set_property PACKAGE_PIN W4 [get_ports {Anode[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Anode[3]}]

## -----
## LD0-LD3 : R7 value (4-bit binary)
## -----
set_property PACKAGE_PIN U16 [get_ports {R7[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {R7[0]}]
set_property PACKAGE_PIN E19 [get_ports {R7[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {R7[1]}]
set_property PACKAGE_PIN U19 [get_ports {R7[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {R7[2]}]
set_property PACKAGE_PIN V19 [get_ports {R7[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {R7[3]}]

## -----
## LD4-LD5 : ALU flags
## -----
set_property PACKAGE_PIN W18 [get_ports Overflow]
    set_property IOSTANDARD LVCMOS33 [get_ports Overflow]
set_property PACKAGE_PIN U15 [get_ports Zero]
    set_property IOSTANDARD LVCMOS33 [get_ports Zero]

## -----
## LD6-LD8 : Feature 1 - Comparator
## -----
set_property PACKAGE_PIN U14 [get_ports Equal]
    set_property IOSTANDARD LVCMOS33 [get_ports Equal]
set_property PACKAGE_PIN V14 [get_ports Less]
    set_property IOSTANDARD LVCMOS33 [get_ports Less]
set_property PACKAGE_PIN V13 [get_ports Greater]
    set_property IOSTANDARD LVCMOS33 [get_ports Greater]

## -----
## LD9-LD10 : Feature 3 - Parity Generator
## -----
set_property PACKAGE_PIN V3 [get_ports Even_Parity]
    set_property IOSTANDARD LVCMOS33 [get_ports Even_Parity]
set_property PACKAGE_PIN W3 [get_ports Odd_Parity]
    set_property IOSTANDARD LVCMOS33 [get_ports Odd_Parity]

## -----
## LD11-LD13 : Feature 4 - Magnitude Detector
## -----
set_property PACKAGE_PIN U3 [get_ports Is_Zero]
    set_property IOSTANDARD LVCMOS33 [get_ports Is_Zero]
set_property PACKAGE_PIN P3 [get_ports Is_Small]
    set_property IOSTANDARD LVCMOS33 [get_ports Is_Small]
set_property PACKAGE_PIN N3 [get_ports Is_Large]
    set_property IOSTANDARD LVCMOS33 [get_ports Is_Large]

```

```
## -----  
## LD14-LD15 : Feature 2 - Multiplier  
## -----  
set_property PACKAGE_PIN P1 [get_ports Mult_OV]  
    set_property IOSTANDARD LVCMOS33 [get_ports Mult_OV]  
set_property PACKAGE_PIN L1 [get_ports Mult_Result]  
    set_property IOSTANDARD LVCMOS33 [get_ports Mult_Result]
```

8 Instructions to operate Advanced Nanoprocessor

8.1 Program (ROM Instructions)

Addr	Assembly	Operation	R7
0	MOVI R7, 3	$R7 = 3$	3
1	MOVI R2, 3	$R2 = 3$	3
2	MUL R7, R2	$R7 = 3 \times 3 = 9$	9
3	MOVI R3, 5	$R3 = 5$	9
4	MOVI R4, 2	$R4 = 2$	9
5	ADD R3, R4	$R3 = 7$ (R7 unchanged)	9
6	NEG R7	$R7 = -9 = 7$ (4-bit)	7
7	JZR R0, 0	Jump to 0	-

8.2 LED Pin Mapping

LED	Pin	Signal	Feature
LD0	U16	R7[0]	LSB
LD1	E19	R7[1]	
LD2	U19	R7[2]	
LD3	V19	R7[3]	MSB
LD4	W18	Overflow	ALU overflow
LD5	U15	Zero	ALU zero
LD6	U14	Equal	Comparator
LD7	V14	Less	Comparator
LD8	V13	Greater	Comparator
LD9	V3	Even_Parity	Parity
LD10	W3	Odd_Parity	Parity
LD11	U3	Is_Zero	Magnitude
LD12	P3	Is_Small	Magnitude (1–7)
LD13	N3	Is_Large	Magnitude (8–15)
LD14	P1	Mult_OV	Multiplier overflow
LD15	L1	Mult_Result	Multiplier bit0

8.3 7-Segment Display

Line	Instruction	Display
0	MOVI R7,3	3
2	MUL R7,R2	9
6	NEG R7	7

Cycle: $3 \rightarrow 9 \rightarrow 7 \rightarrow \dots$

8.4 Step-by-Step Demo

8.4.1 Step 1 – Reset

- Display: 0
- R7 = 0
- LD11 (Zero) ON
- LD9 (Even Parity) ON

8.4.2 Step 2 – MOVI R7,3

- Display: 3
- LD0, LD1 ON
- LD12 (Small) ON

8.4.3 Step 3 – MUL R7,R2

- Display: 9
- LD15 ON (result bit0)
- LD13 ON (Large)

8.4.4 Step 4 – ADD R3,R4

- Display: 9
- LD8 ON (Greater: $5 \geq 2$)

8.4.5 Step 5 – NEG R7

- Display: 7
- LD10 ON (Odd parity)
- LD12 ON (Small)

8.5 LED Pattern Summary

Format: LD15 $\rightarrow$ LD0

State	LED Pattern	Display
Reset	off off off off ON off ON off off ON off off off off off off	0
After MUL	ON off ON off off off ON off off ON off off ON off off ON	9
After ADD	ON off ON off off off ON ON off off off off ON off off ON	9
After NEG	off off off ON off ON off off ON off off off off ON ON ON	7

8.6 Feature Summary

8.6.1 Comparator

- LEDs: LD6, LD7, LD8
- Example: $5 > 2 \Rightarrow$ LD8 ON

8.6.2 Multiplier

- LEDs: LD14, LD15
- Example: $3 \times 3 = 9$, LD15 ON

8.6.3 Parity Generator

- LEDs: LD9 (Even), LD10 (Odd)
- $9 \rightarrow$ Even, $7 \rightarrow$ Odd

8.6.4 Magnitude Detector

- LEDs: LD11, LD12, LD13
- $0 \rightarrow$ Zero, $3 \rightarrow$ Small, $9 \rightarrow$ Large

9 Problems Faced and Solutions

9.1 Incorrect Load_sel Logic in Instruction Decoder

Problem: R7 remained at 0 throughout simulation. The `MOVI` instruction incorrectly set:

$$\text{Load_sel} = 0$$

which routed the ALU output to the register instead of the immediate value. Similarly, the `ADD` instruction used the opposite logic.

Solution:

- Corrected the decoder case statement
- `MOVI` now sets:

```
Load_sel <= '1';
```

- `ADD` and `NEG` now set:

```
Load_sel <= '0';
```

After correction, R7 updated correctly:

$$1 \rightarrow 3 \rightarrow 6$$

9.2 Slow Clock Too Slow for Simulation

Problem: Even after fixing the decoder, R7 did not change during simulation. The slow clock divider used:

```
count(25)
```

which produced a clock period of approximately 671 ms.

At a simulation time of only 1000 ns, no slow clock edge occurred, so no instructions executed.

Solution:

- Changed the slow clock divider to:

```
count(1)
```

for simulation purposes

- This produced a clock period of approximately 40 ns
- Before programming the FPGA board, reverted back to:

```
count(25)
```

This allowed instructions to execute visibly on the BASYS3 board at approximately 0.67 seconds per instruction.

9.3 DRC Errors NSTD-1 and UCIO-1 (Advanced Version)

Problem: Vivado failed to generate the bitstream because:

- `Mult_Result[3:1]` had no pin assignments
- Missing `LOC` and `IOSTANDARD` constraints in the XDC file

Vivado requires all top-level ports to have valid physical pin mappings.

Solution:

- Temporarily changed:

```
STD_LOGIC_VECTOR(3 downto 0)
```

to:

```
STD_LOGIC
```

so only bit 0 was exposed

- No LEDs left for further assignments.

10 Resource Optimizations

10.1 `with-select` for 2-Way Multiplexers

Optimization: Concurrent `with-select` assignments were used instead of process blocks for implementing 2-way multiplexers.

Advantages:

- Synthesizes into a single LUT
- Avoids unnecessary sequential logic
- Eliminates the risk of latch inference
- Produces cleaner combinational hardware

Implementation:

```
with Load_sel select
  Q <= Adder_Sub_Out when '0',
      Imd_Value       when others;
```

10.2 `case-when` for 8-Way Multiplexer

Optimization: A `case` statement was used instead of an `if-elsif` chain for the 8-way multiplexer implementation.

Advantages:

- Synthesizes into a balanced multiplexer tree
- Reduces LUT depth
- Improves timing performance
- Avoids priority encoder behavior

Observation:

- `case` statements create parallel selection logic
- `if-elsif` chains synthesize as priority logic with longer propagation paths

10.3 `to_integer` Indexing for 3-to-8 Decoder

Optimization: Used:

```
to_integer(unsigned(I))
```

to directly index the decoder output vector instead of using multiple explicit comparisons.

Advantages:

- Produces a compact decode network
- Minimizes redundant comparison logic
- Reduces LUT utilization
- Improves readability and scalability

Implementation Concept:

```
Y(to_integer(unsigned(I))) <= '1';
```